

CHAPTER 6

Digital CMOS Logic Design

KEY TOPICS

- Digital logic design
- CMOS logic design
- CMOS design methodology
- Design of CMOS inverter
- Design of two-input NAND gate
- Design of two-input NOR gate
- Classification of CMOS digital logic circuit
- Combinational logic circuit
- Sequential logic circuit
- Pseudo-nMOS logic
- CMOS transmission gate
- Dynamic CMOS logic
- Domino CMOS logic
- NORA CMOS logic
- Zipper CMOS logic
- True single-phase clock dynamic CMOS logic
- Pass transistor logic
- Complementary pass transistor logic
- Voltage bootstrapping
- Differential CMOS logic
- Adiabatic logic
- Dual-threshold CMOS logic

6.1 Introduction

In this chapter, we introduce the digital logic circuit design using CMOS transistors, which are most popular because of low power dissipation and less area requirement compared to any other logic circuits. It is worthwhile mentioning that almost 90% of the total semiconductor devices are fabricated using silicon CMOS technology. Silicon (Si) is the most suitable semiconductor material for VLSI circuit fabrication because of its native oxide SiO_2 . CMOS logic is a combination of nMOS and pMOS logic. The nMOS and pMOS transistors are both functionally and structurally complement to each other. Hence, the combination of nMOS and pMOS is known as complementary MOS or CMOS.

6.2 Digital Logic Design

In digital logic design, there are three primary logic operations: (a) NOT, (b) OR, and (c) AND. Using these three primary logics, any other logic such as NAND, NOR, XOR, or XNOR can be derived.

In NOT logic, if the input (A) is TRUE, the output (F) is FALSE; and if the input (A) is FALSE, the output (F) is TRUE. The NOT logic can be realized using a simple circuit as shown in Fig. 6.1(a). When the switch S_1 is ON, the bulb will not glow. When the switch S_1 is OFF, the bulb will glow.

In OR logic, if both the inputs are FALSE, the output is FALSE. Otherwise, the output is TRUE. The OR logic can be realized using two switches S_1 and S_2

connected in parallel, as shown in Fig. 6.1(b). When both switches are OFF, the bulb will not glow; otherwise, the bulb will glow.

In AND logic, if both the inputs are TRUE, the output is TRUE. Otherwise, the output is FALSE. The AND logic can be realized using two switches S1 and S2 connected in series as shown in Fig. 6.1(c).

The mechanical switches shown in Fig. 6.1 can be replaced by MOS transistors as the MOS transistor behaves as a switch when it is operated between the cut-off and saturation regions.

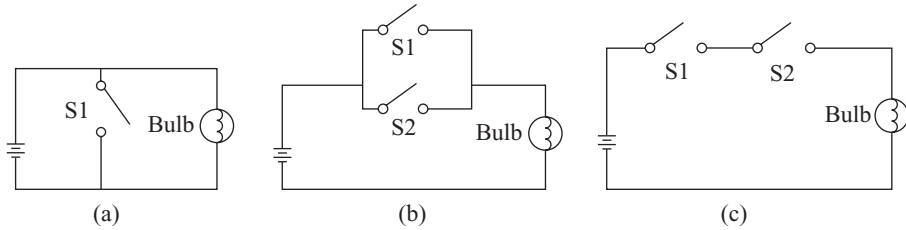


Fig. 6.1 (a) Realization of NOT logic using one switch; (b) realization of OR logic using two switches; (c) realization of AND logic using two switches

As shown in Fig. 6.2(a), an nMOS transistor can be modelled as a switch connected between the drain (D) and source (S) and the switch is controlled by gate (G). When the gate is logic high (H), the nMOS is ON and the switch is closed, and D is connected to S [see Fig. 6.2(b)]. When the gate is logic low (L), the nMOS is OFF and the switch is open, and D is disconnected from S [see Fig. 6.2(c)].

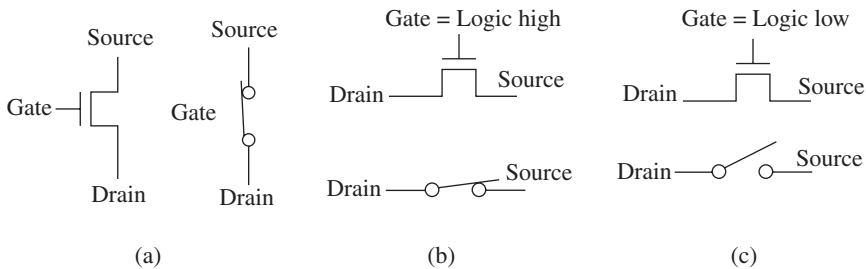


Fig. 6.2 (a) nMOS transistor and its switch model; (b) ON nMOS modelled as closed switch; (c) OFF nMOS modelled as open switch

Similarly, a pMOS can also be modelled as a switch as shown in Figs 6.3(a)–6.3(c).

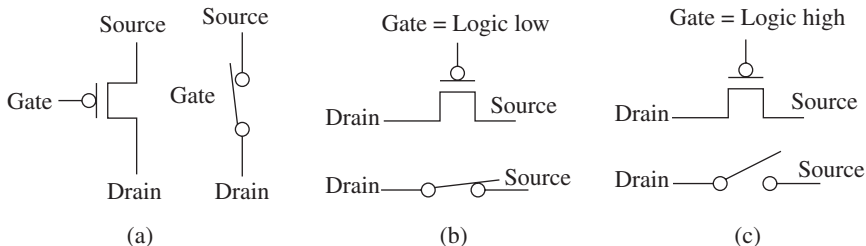


Fig. 6.3 (a) pMOS transistor and its switch model; (b) ON pMOS modelled as closed switch; (c) OFF pMOS modelled as open switch

In reality, an nMOS can pass logic low perfectly but cannot pass logic high perfectly. On the other hand, a pMOS can pass logic high perfectly but cannot pass logic low perfectly. This is illustrated in Figs 6.4(a) and (b).

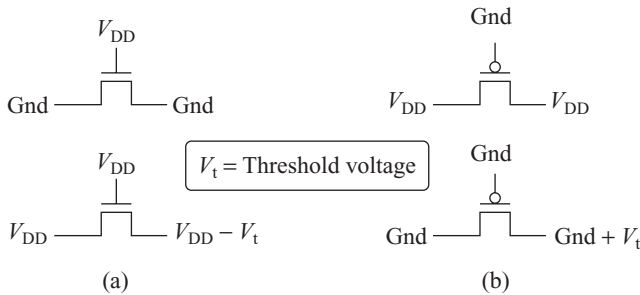


Fig. 6.4 (a) Logic level passing through nMOS; (b) logic level passing through pMOS

The logic degradation mechanism is explained in the following text with the help of a circuit shown in Fig. 6.5.

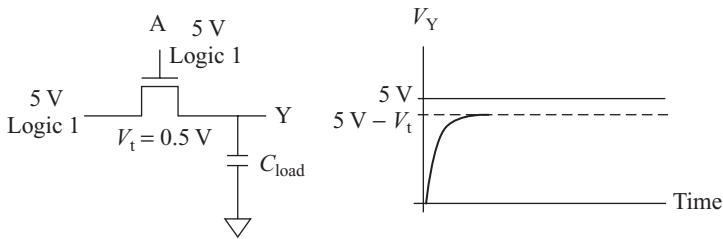


Fig. 6.5 Transfer of logic 1 through an nMOS transistor

Let us consider the case of transfer of logic 1 through an nMOS transistor having threshold voltage $V_t = 0.5$ V, as shown in Fig. 6.5. To make the nMOS transistor ON, we apply logic 1 ($= 5$ V) to its gate. At the input, we apply logic 1 to transfer it to the output. Assume that the load capacitor is initially at logic 0 ($= 0$ V). Hence, gate-to-source (V_{GS}) potential difference is 5 V and gate-to-drain (V_{GD}) potential difference is 0 V. Therefore, the ON condition for the nMOS transistor is satisfied. Current flows through the nMOS transistor from the drain to the source and the load capacitor slowly gets charged. As the load capacitor gets charged, the potential at node Y (V_Y) increases. As V_Y increases, V_{GS} decreases. When V_Y reaches $(5 \text{ V} - V_t)$, V_{GS} is just V_t . Under this condition, the nMOS transistor is still ON. But any further increase in V_Y causes V_{GS} to become less than V_t , and makes the nMOS transistor OFF. So no further increase in V_Y is possible. Hence, the maximum voltage at the output is $5 \text{ V} - V_t$, and, thus we get a degraded logic 1 at the output.

In the case of logic 0 transfer through an nMOS transistor, the gate and drain terminals are at 5 V and the source terminal is at 0 V. Note that in this case input node is source and the output node is drain that is initially charged to logic 1. Readers must keep in mind that the source and drain are mutually interchangeable due to symmetrical structure of the MOSFET. Now V_{GS} is 5 V and V_Y discharges through the

nMOS transistor keeping V_{GS} at 5 V all the time. So the load capacitor can be fully discharged making V_Y to become 0 V. Hence, an nMOS transistor can pass logic 0 perfectly.

Similarly, we can explain that a pMOS transistor passes logic 1 perfectly but passes degraded logic 0. Hence, the bottom line is that the pMOS should be used for pulling a node to logic high, and nMOS should be used for pulling a node to logic low.

6.3 CMOS Logic Design

Any Boolean logic function (F) has possible values: either logic 0 or logic 1. For some of the input combinations, $F = 1$ and for all other input combinations, $F = 0$. So in general, any Boolean logic function can be realized using a structure as shown in Fig. 6.6. The switch S1 is closed and the switch S2 is open for input combinations that produce $F = 1$. The switch S1 is open and the switch S2 is closed for other input combinations that produce $F = 0$.

As shown in Fig. 6.6, the output (F) is either connected to V_{DD} or to the ground, where the logic 0 is represented by the ground and the logic 1 is represented by V_{DD} . So the basic requirement of digital logic design is to implement the pull-up switch (S1) and the pull-down switch (S2). As the pMOS transistors can pass logic 1 perfectly, they are used in pull-up switch realization. Similarly, as the nMOS transistors can pass logic 0 perfectly, they are used in pull-down switch realization.

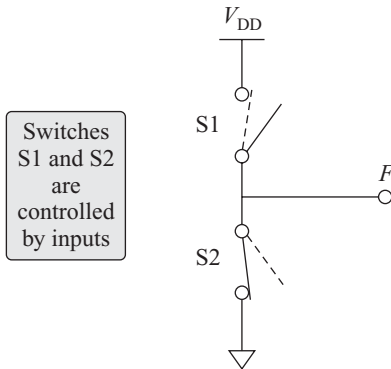


Fig. 6.6 Realization of any Boolean logic function (F)

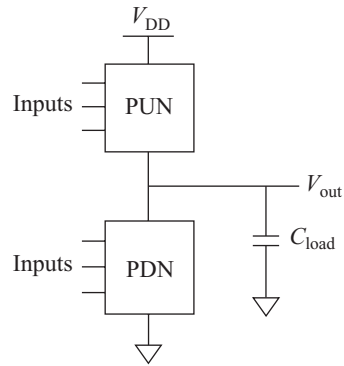


Fig. 6.7 A general CMOS logic circuit

A generalized CMOS logic circuit consists of a pull-up network (PUN) and a pull-down network (PDN), as shown in Fig. 6.7. The PUN comprises pMOSFETs and the PDN comprises nMOSFETs. Depending on the applied input logic, the PUN connects the output node to V_{DD} , and the PDN connects the output node to the ground.

6.4 CMOS Design Methodology

The basic CMOS design methodology involves three steps:

- Given the Boolean expression, take its complement
- Design PDN by realizing
 - AND terms using series-connected nMOSFETs
 - OR terms using parallel-connected nMOSFETs
- Design PUN just reverse (or dual) of the PDN

6.5 Design of CMOS Inverter (NOT) Gate

A CMOS inverter is the simplest logic circuit that uses one nMOS and one pMOS transistor. The nMOS is used in PDN and the pMOS is used in the PUN, as shown in Figs 6.8(a)–(c).

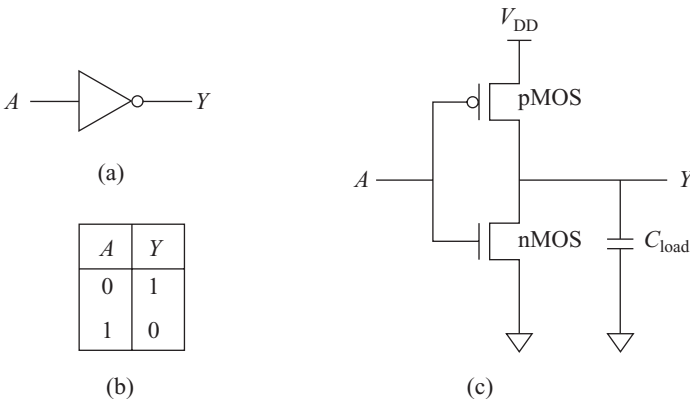


Fig. 6.8 (a) Symbol for inverter; (b) truth table of inverter; (c) CMOS realization of inverter

Operation When input is low, the nMOS is OFF and the pMOS is ON. Hence, the output is connected to V_{DD} through pMOS. When the input is high, the nMOS is ON and the pMOS is OFF. Hence, the output is connected to the ground through nMOS. We can connect a capacitor at the output node as shown in Fig. 6.8 to represent the load seen by the inverter. The load capacitor is charged to V_{DD} through pMOS when the input is low and is discharged to the ground through nMOS when the input is high.

6.6 Design of Two-input NAND Gate

To illustrate the design methodology, let us consider a simple example of a two-input NAND gate design. The two-input NAND function is expressed by

$$Y = \overline{A \cdot B} \tag{6.1}$$

Step 1: Take complement of Y

$$\overline{\overline{Y}} = \overline{\overline{A \cdot B}} = A \cdot B \tag{6.2}$$

Step 2: Design the PDN

In this case, there is only one AND term. So there will be two nMOSFETs in series, as shown in Fig. 6.9.

Step 3: Design the PUN

In PUN, there will be two pMOSFETs in parallel, as shown in Fig. 6.10.

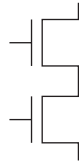


Fig. 6.9 Pull-down network comprising nMOSFETs

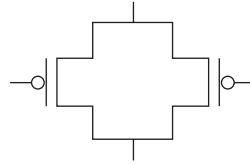
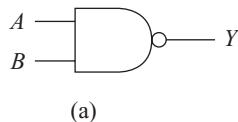


Fig. 6.10 Pull-up network comprising pMOSFETs

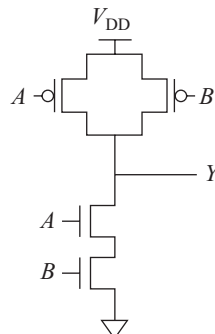
Now join the PUN and PDN as shown in Fig. 6.11(c). Note that we have realized $\bar{Y}$, rather than Y because the inversion is automatically provided by the nature of the CMOS circuit operation.



(a)

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

(b)



(c)

Fig. 6.11 Two-input NAND gate: (a) symbol; (b) truth table; (c) CMOS realization

Operation When $A = 0$ and $B = 0$, both the nMOS transistors are OFF and both pMOS transistors are ON. Hence, the output is connected to V_{DD} and we get logic high at the output.

When $A = 1$ and $B = 0$, the upper nMOS is ON and lower nMOS is OFF. So, output cannot be connected to the ground. Under this condition, left pMOS is OFF but right pMOS is ON. Hence, the output is connected to V_{DD} , and we get logic high at the output.

When $A = 0$ and $B = 1$, the upper nMOS is OFF and lower nMOS is ON. So, output cannot be connected to ground. Under this condition, left pMOS is ON but right pMOS is OFF. Hence, the output is connected to V_{DD} , and we get logic high at the output.

When $A = 1$ and $B = 1$, both nMOS transistors are ON and both pMOS transistors are OFF. Hence, the output is connected to the ground, and we get logic low at

the output. This is illustrated in Figs 6.11(a) and (c). This proves by the truth table of NAND gate shown in Fig. 6.11(b).

6.7 Design of Two-Input NOR Gate

Let us consider another example of a two-input NOR gate. The two-input NOR function is expressed by

$$Y = \overline{A + B}$$
 (6.3)

Step 1: Take complement of Y

$$\overline{Y} = \overline{\overline{A + B}} = A + B$$
 (6.4)

Step 2: Design the PDN

Here, there is only one OR term. Hence, there will be two nMOSFETs connected in parallel, as shown in Fig. 6.12.

Step 3: Design the PUN

In the PUN, two pMOSFETs will be connected in series, as shown in Fig. 6.13.

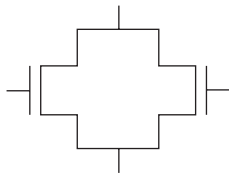


Fig. 6.12 Pull-down network comprising nMOSFETs

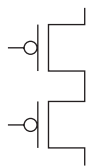
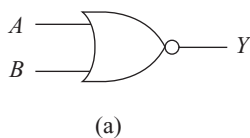


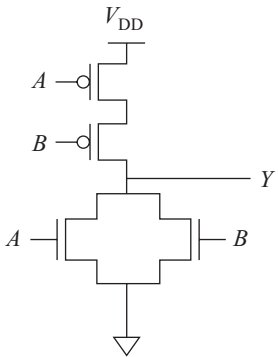
Fig. 6.13 Pull-up network comprising pMOSFETs

Now, join the PUN and PDN as shown in Fig. 6.14(c).



A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

(b)



(c)

Fig. 6.14 Two-input NOR gate: (a) symbol; (b) truth table; (c) CMOS realization

Operation When $A = 0$ and $B = 0$, both nMOS transistors are OFF and both pMOS transistors are ON. Hence, the output is connected to V_{DD} and we get logic high at the output.

When $A = 1$ and $B = 0$, the upper pMOS is OFF and lower pMOS is ON. So, output cannot be connected to the V_{DD} . Under this condition, left nMOS is ON and right nMOS is OFF. Hence, the output is connected to the ground and we get logic low at the output.

When $A = 0$ and $B = 1$, the upper pMOS is ON and lower pMOS is OFF. So, output cannot be connected to V_{DD} . Under this condition, left nMOS is OFF and right nMOS is ON. Hence, the output is connected to the ground and we get logic low at the output.

When $A = 1$ and $B = 1$, both nMOS transistors are ON and both pMOS transistors are OFF. Hence, the output is connected to V_{DD} and we get logic low at the output. This proves the truth table of NOR gate as shown in Fig. 6.14(b).

6.8 Classification of CMOS Digital Logic Circuit

CMOS logic circuits are mainly classified into two categories as follows:

- Combinational logic circuit
- Sequential logic circuit

In the combinational logic circuit, the output is determined by the present logic inputs. However, in the sequential logic circuit, the output is determined by the present inputs and past outputs. The examples of combinational logic circuits are Inverter, NAND gate, NOR gate, multiplexer, demultiplexer, decoder, encoder, half-adder, full-adder, etc. The examples of sequential logic circuits are flip-flops, latches, registers, counters, etc.

There are other CMOS design styles as given below:

- CMOS transmission logic
- Complementary pass-transistor logic
- Dynamic CMOS logic
- Domino CMOS logic
- NORA CMOS logic
- Zipper CMOS logic

6.9 Combinational Logic Circuit

In this section, we discuss how the combinational logic circuit is designed with a few examples. As we have discussed before, any CMOS logic circuit consists of a PUN and a PDN. The PUN comprises only pMOSFETs, and the PDN comprises of only nMOSFETs. There is a fundamental reason why pMOSFETs are used in PUN and the nMOSFETs in PDN (also explained before).

6.9.1 Design of Complex Logic Circuit

Let us now design a complex Boolean function given below:

$$Y = \overline{A(B+C)} + DE \quad (6.5)$$

Step 1: Take complement of Y

$$\bar{Y} = \overline{A(B+C) + DE} = A(B+C) + DE \quad (6.6)$$

Step 2: Design of PDN

Let $Z = B = C$, then Eqn (6.6) becomes

$$\bar{Y} = AZ + DE \quad (6.7)$$

Now, $Z (=B = C)$ is realized by two nMOSFETs connected in parallel. Let us call this as sub-logic realizing Z . In Eqn (6.7), there are two AND terms which are ORed. AZ is realized by one nMOSFET connected in series with sub-logic realizing Z . The term DE is realized by two nMOSFETs connected in series. Finally, two sub-logics realizing AZ and DE are connected in parallel.

Step 3: Design of PUN

We just do the reverse connection of nMOSFETs to realize the PUN using pMOSFETs. The complete CMOS logic is shown in Fig. 6.15.

The Boolean function given in Eqn (6.5) can also be realized, as shown in Fig. 6.16. In this case, we expand Eqn (6.6) into:

$$\bar{Y} = AB + AC + DE \quad (6.8)$$

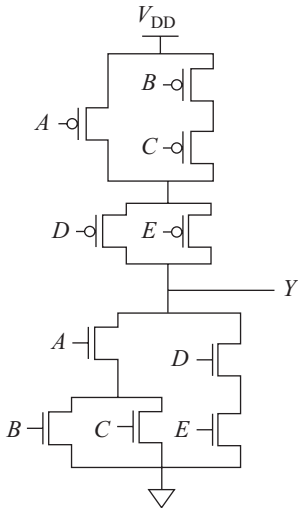


Fig. 6.15 CMOS logic realizing the Boolean function given in Eqn (6.5)

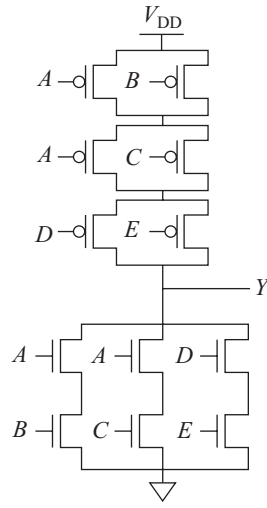


Fig. 6.16 CMOS logic realizing the Boolean function given in Eqn (6.8)

Comparing the realizations shown in Figs 6.15 and 6.16, we can see that in Fig. 6.16, we need more transistors than in Fig. 6.15. This means that the CMOS logic realization with the minimum number of transistors requires judicious function optimization.

6.9.2 Design of AND-OR-INVERT and OR-AND-INVERT Gate

Any Boolean function can be expressed either in the SOP (sum-of-products) or POS (product-of-sum) form. The AND-OR-INVERT (AOI) gate is suitable for realizing functions in the SOP form. Here a function is expressed as a summation (OR) of product (AND) terms. For example, a function X in the SOP form with a complement can be written as

$$X = \overline{AB + CD + EFG} \quad (6.9)$$

The AOI implementation of the function X is shown in Fig. 6.17. Each AND term is implemented by a series combination of nMOS transistors. The OR operation between the AND terms is realized by a parallel connection of each series-connected nMOS transistors subcircuit. The PUN is implemented as a dual of an nMOS network.

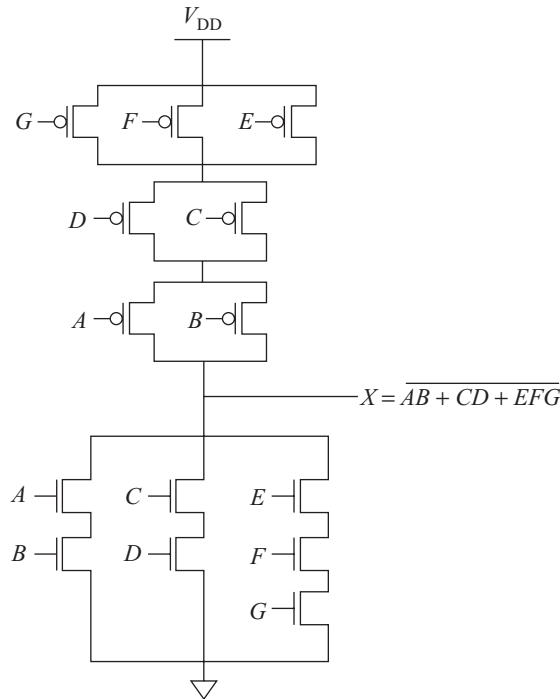


Fig. 6.17 AOI gate realization using CMOS logic

Similarly, a function expressed in the POS form with a complement can be written as

$$Y = \overline{(A + B)(C + D)(E + F + G)} \quad (6.10)$$

The OR-AND-INVERT (OAI) realization of the function Y is shown in Fig. 6.18. Each OR term is implemented by a parallel combination of nMOS transistors. The AND operation between the OR terms is realized by the series connection of each parallel-connected nMOS. The PUN is implemented as a dual of the nMOS network.

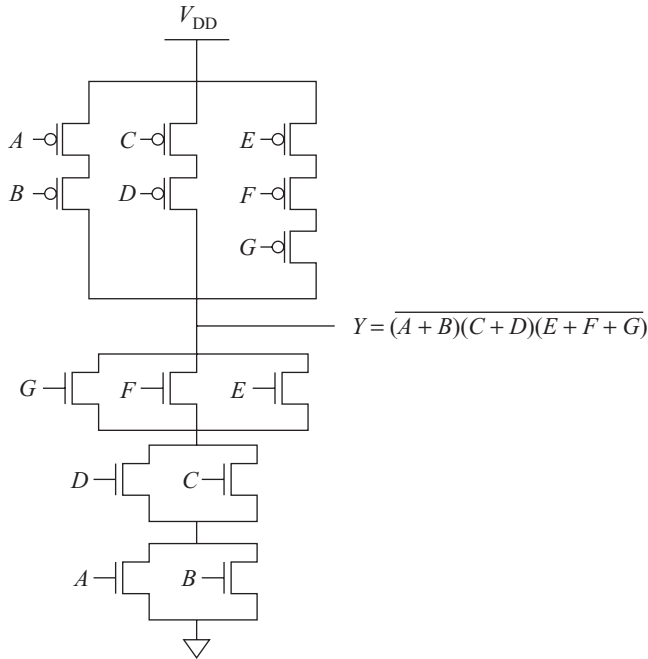


Fig. 6.18 OAI gate realization using CMOS logic

6.9.3 Design of XOR Gate

The Boolean expression for two-input XOR gate is given by

$$Y = A\bar{B} + \bar{A}B \quad (6.11)$$

Step 1: Take complement of Y

$$\bar{Y} = \overline{A\bar{B} + \bar{A}B} = \overline{A\bar{B}} \times \overline{\bar{A}B} = (\bar{A} + B) \times (A + \bar{B}) = \bar{A}\bar{B} + AB \quad (6.12)$$

Step 2: Design of PDN

Here, there are two AND terms and one OR term. Each of the AND term is realized by two series-connected nMOSFETs. The OR term is realized by parallel connection of two series-connected nMOSFETs realizing AND terms.

Step 3: Design of PUN

In PUN, we need four pMOSFETs connected in a reverse manner. The complete CMOS circuit realizing two-input XOR gate is shown in Fig. 6.19.

Note: In realizing the two-input XOR gate, we have assumed that the inputs are available both in the normal and complemented form. This is a valid assumption because in a full-chip design, the inputs might come from the output of flip-flop or latch where both the normal and complemented forms of the output (Q and $\bar{Q}$) are available. In case the complemented input is not available, an inverter is to be used to generate the complemented input.

6.9.4 Design of Half-adder Circuit

A half-adder circuit has two inputs and two outputs: Sum and Carry. The truth table of a half-adder circuit is shown in Table 6.1.

Table 6.1 Truth table of half-adder

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Boolean expressions for the sum and carry are given by

$$\text{Sum} = A\bar{B} + \bar{A}B \quad (6.13)$$

$$\text{Carry} = AB \quad (6.14)$$

CMOS realization of Sum is shown in Fig. 6.19 and the Carry is shown in Fig. 6.20.

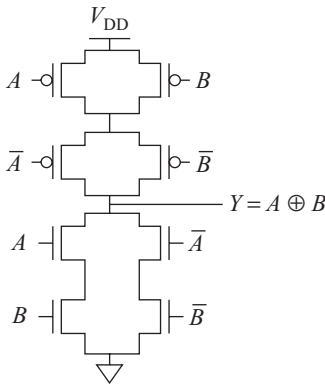


Fig. 6.19 CMOS realization of two-input XOR gate

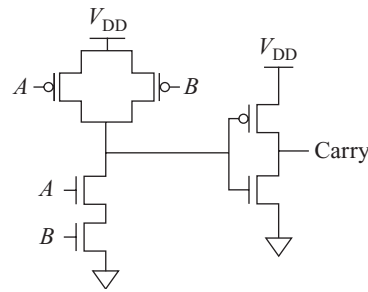


Fig. 6.20 CMOS realization of carry function of half-adder circuit

6.9.5 Design of Full-adder Circuit

A full-adder circuit has three inputs: A , B , and Carry-in (C_{in}) and two outputs: Sum and Carry-out (C_{out}). The truth table of a full-adder circuit is shown in Table 6.2.

Boolean expressions for the Sum and C_{out} are given by

$$\text{Sum} = \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in} \quad (6.15)$$

$$C_{out} = AB + AC_{in} + BC_{in} \quad (6.16)$$

As shown in Table 6.3, taking $\bar{C}_{out}$ as input, we could simplify the expression for Sum as

$$\begin{aligned} \text{Sum} &= \bar{A}\bar{C}_{out} + B\bar{C}_{out} + C_{in}\bar{C}_{out} + ABC_{in} \\ &= (A + B + C_{in})\bar{C}_{out} + ABC_{in} \end{aligned} \quad (6.17)$$

Table 6.2 Truth table of full-adder

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 6.3 Truth table of full-adder (modified to include $\overline{\text{Cout}}$)

A	B	Cin	Cout	$\overline{\text{Cout}}$	Sum
0	0	0	0	1	0
0	0	1	0	1	1
0	1	0	0	1	1
0	1	1	1	0	0
1	0	0	0	1	1
1	0	1	1	0	0
1	1	0	1	0	0
1	1	1	1	0	1

The complete CMOS realization of full-adder is shown in Fig. 6.21.

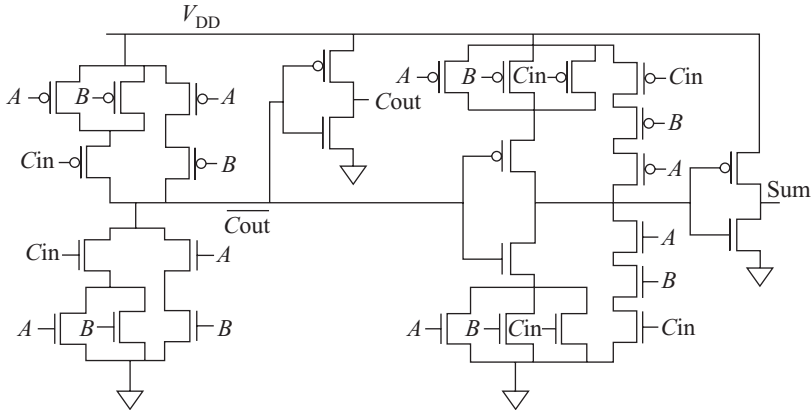


Fig. 6.21 CMOS realization of full-adder circuit

Example 6.1 Design a 2×2 input AND–OR gate to realize the following Boolean function:

$$Y = AB + CD$$

Solution To realize the Boolean function, first we find out the complement of the function.

Complement of the function, $\overline{Y} = \overline{AB + CD} = \overline{AB} \times \overline{CD} = (\overline{A} + \overline{B}) \times (\overline{C} + \overline{D})$. We can see $\overline{Y}$ has two OR terms and one AND term. Realize each OR term by parallel connection of two nMOS. Connect them in series to realize the AND term as shown in Fig. 6.22.

Example 6.2 Realize a tri-state buffer.

Solution A tri-state buffer can be realized as shown in Fig. 6.23(c).

The circuit implementation as shown in Fig. 6.23(d) is not recommended as it suffers from dynamic charge sharing problem.

Example 6.3 Design a 2:1 multiplexer using static CMOS logic.

Solution A 2:1 multiplexer can be realized, as shown in Fig. 6.24(c).

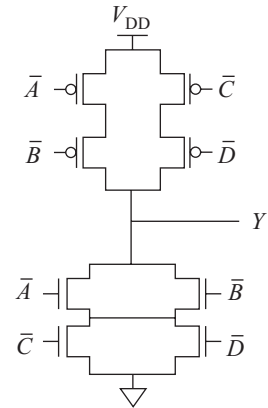


Fig. 6.22 Schematic to realize function $Y = AB + CD$

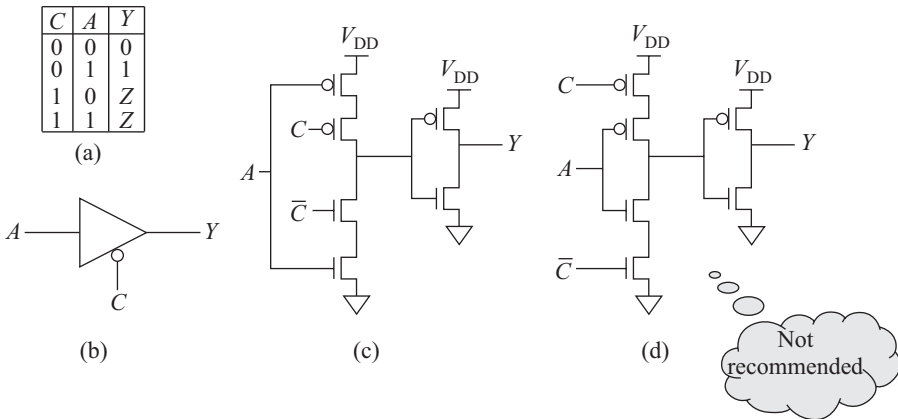


Fig. 6.23 Tri-state buffer: (a) truth table; (b) symbol; (c) and (d) CMOS realization

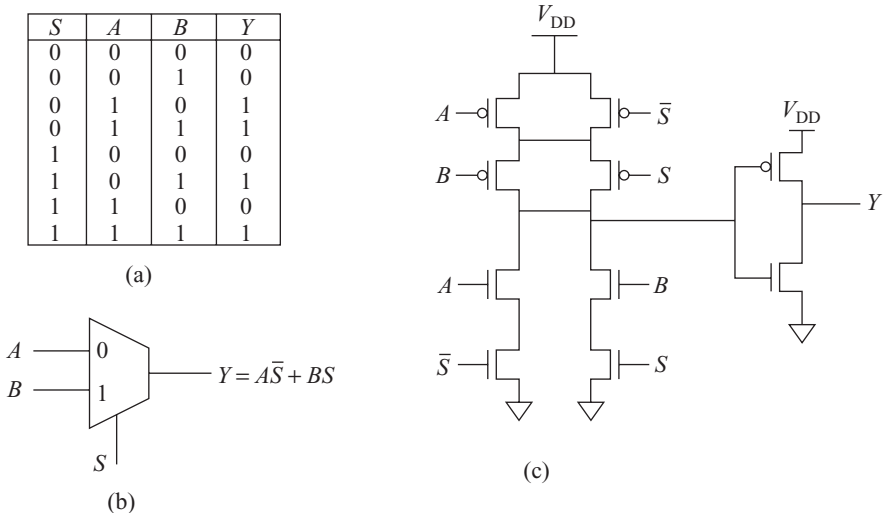


Fig. 6.24 2:1 Multiplexer: (a) truth table; (b) symbol; (c) CMOS realization

6.10 Sequential Logic Circuit

In the combinational logic circuits, the output is determined by the input logic levels. The combinational circuit lacks the storing capability of any previous events. In the sequential logic circuits, the output is determined by the present input logic level, as well as the past output logic level. In the sequential circuit, there is a feedback connection between the output and the input.

The generalized structure of a sequential circuit is shown in Fig. 6.25. It consists of a combinational logic block and a memory circuit in the feedback path.

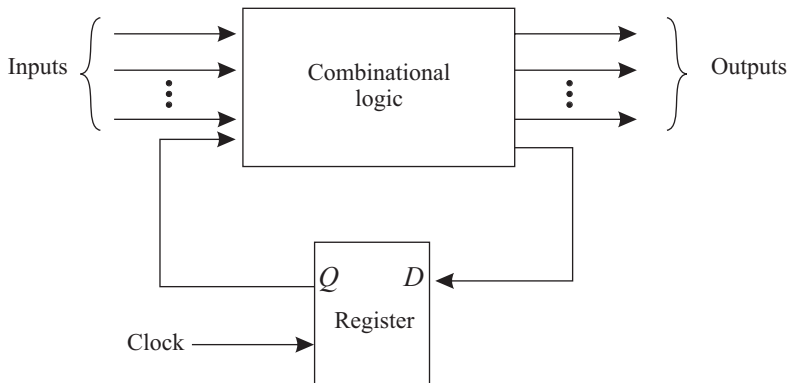


Fig. 6.25 Schematic of a sequential logic circuit

The feedback path contains the register that has the bit storing capability. The sequential logic circuit is also known as *regenerative* logic circuit. The regenerative circuits are formed with positive feedback. They are classified into three following types:

- Bistable circuits—have two stable states.
- Monostable circuits—have only one stable state.
- Astable circuits—have no stable state, rather they oscillate between two states.

Bistable circuits are the most commonly used circuits. Latches, flip-flops, registers, and memory are the examples of bistable circuits.

6.10.1 Working Principle of Bistable Circuits

Let us consider the basic bistable circuit consisting of two cross-coupled inverters (Inv1 and Inv2) as shown in Fig. 6.26.

The input and output of the first inverter (Inv1) are V_{in1} and V_{out1} . The voltage transfer characteristic (VTC) of the first inverter is shown in Fig. 6.27(a).

The input and output of the second inverter (Inv2) are V_{in2} and V_{out2} . The VTC of the second inverter is shown in Fig. 6.27(b). In this plot, the axes for input and output voltages are just interchanged.

When two inverters are connected back-to-back, we can write $V_{in2} = V_{out1}$ and $V_{in1} = V_{out2}$. In order to find out the operating points of the circuit shown in

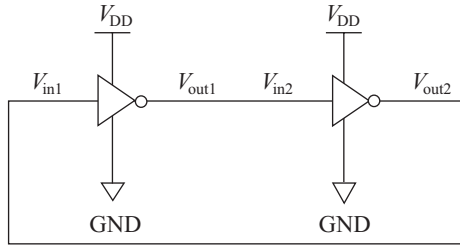


Fig. 6.26 Basic bistable circuit with two back-to-back connected inverters

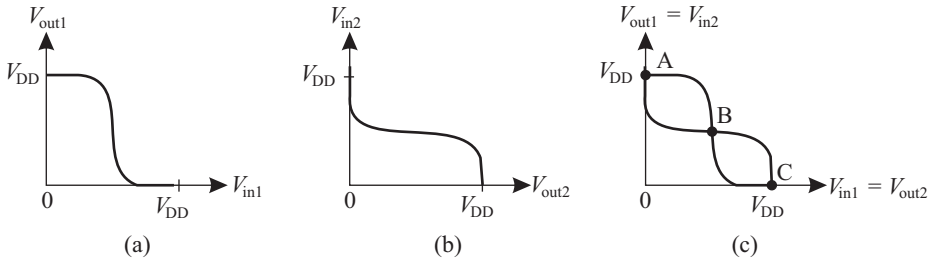


Fig. 6.27 Voltage transfer characteristics

Fig. 6.26, let us now combine the two VTCs as shown in Fig. 6.27(c). As shown in Fig. 6.27(c), there are three operating points: *A*, *B*, and *C*.

Now, let us suppose that the operating point is at *A*. That is, the input of the first inverter $V_{in1} = 0$, therefore, $V_{out1} = 1 = V_{in2}$ and consequently $V_{out2} = 0 = V_{in1}$. Hence, the circuit remains stable at this operating point. Any small deviation from the operating point does not cause any change in the outputs.

Similarly, let us suppose that the operating point is at *C*. That is, the input of the first inverter $V_{in1} = 1$, therefore, $V_{out1} = 0 = V_{in2}$ and consequently $V_{out2} = 1 = V_{in1}$. Hence, the circuit remains stable at this operating point. In this case also, any small deviation from the operating point does not cause any change in the outputs.

Now, let us suppose that the operating point is at *B*. Now, say the input V_{in1} is slightly reduced by an amount ΔV due to some noise. The output V_{out1} and the input V_{in2} will be increased which will cause V_{out2} to decrease and so to V_{in1} . This will continue till V_{in1} reaches 0 and V_{out1} reaches V_{DD} . Similarly, we can show that at the operating point *B*, a small increase in V_{in1} will continue to decrease V_{out1} till it reaches 0 and V_{in1} reaches V_{DD} . Therefore, at the operating point *B*, a small deviation in the voltage level will move the operating point to either *A* or *C*.

Hence, the circuit has two stable operating points *A* and *C* and *B* is an unstable operating point. Since the circuit has two stable operating points and hence two stable states (either 0 or V_{DD}), the circuit is called *bistable circuit*.

6.10.2 Design of SR Latch Circuit

In the SR latch circuit, there are two inputs: *S* (Set) and *R* (Reset) and two complementary outputs: Q and $\bar{Q}$. The input and output relationship is described in Table 6.4.

Table 6.4 Truth table of the NOR-based SR latch circuit

S	R	Q_{n+1}	$\overline{Q}_{n+1}$	Operation
0	0	Q_n	$\overline{Q}_n$	Hold
1	0	1	0	Set
0	1	0	1	Reset
1	1	0	0	Not allowed

The gate-level schematic of the SR latch circuit is shown in Fig. 6.28.

Figure 6.29 shows the CMOS realization of the NOR-based SR latch circuit.

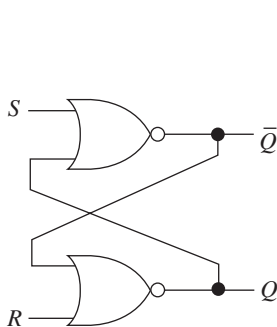


Fig. 6.28 Gate-level schematic of SR latch realized using NOR gate

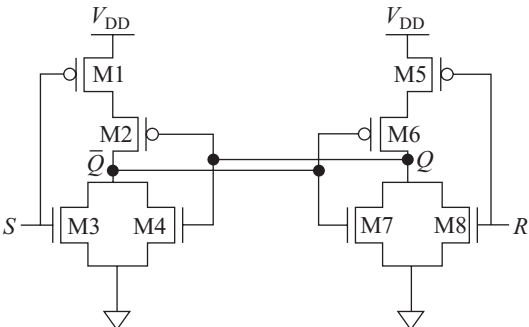


Fig. 6.29 CMOS realizations of NOR-based SR latch circuit

The next state of SR latch can be expressed as

$$Q_{n+1} = S + \overline{R}Q_n \quad (\text{for } SR = 0) \tag{6.18}$$

The condition $SR = 0$ implies that both S and R cannot be 1 at the same time. An equation that expresses the next state value of a latch in terms of its present state and inputs is referred to as a ‘next-state equation’ or a ‘characteristic equation’.

Operation When $S = 0$ and $R = 0$, the nMOS transistors M3 and M8 are OFF. The pMOS transistors M1 and M5 are ON. This cannot determine the output logic. Let us assume $\overline{Q} = 0$, then M2 is ON and M4 is OFF. Thus, the node $\overline{Q}$ is connected to V_{DD} , i.e., $\overline{Q} = 1$. As $\overline{Q} = 1$, M7 is ON and M6 is OFF. Thus, node Q is connected to the ground, i.e., Q remains at logic 0 state. If we assume $\overline{Q} = 1$, then M4 is ON and M2 is OFF. Thus, node $\overline{Q}$ is connected to the ground, i.e., $\overline{Q} = 0$. As $\overline{Q} = 0$, M6 is ON and M7 is OFF. Thus, node Q is connected to V_{DD} , i.e., Q remains at logic 1 state. This proves the first row of the truth table of SR latch.

When $S = 0$, $R = 1$, the transistor M8 is ON. Thus, irrespective of other input and past output conditions, the node Q is connected to the ground, i.e., Q is in logic 0 state.

When $S = 1$ and $R = 0$, the transistor M3 is ON. Thus, irrespective of other input and past output conditions, the node $\overline{Q}$ is connected to the ground, i.e., $\overline{Q}$ is in logic 0 state. This makes M5 and M6 ON; thus, node Q is connected to V_{DD} , i.e., Q is in logic 1 state.

If S and R inputs are zero at the same time, both the outputs will be zero. Hence, this combination is not allowed.

The SR latch circuit can also be realized using NAND gates as shown in Fig. 6.30.

Table 6.5 shows the truth table of NAND-based SR latch circuit.

Figure 6.31 shows the CMOS realization of the NAND-based SR latch circuit.

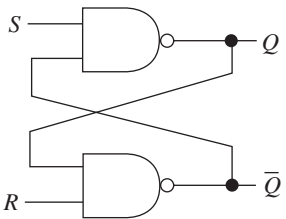


Fig. 6.30 Gate-level schematic of SR latch realized using NAND gate

Table 6.5 Truth table of the NAND-based SR latch circuit

S	R	Q_{n+1}	$\bar{Q}_{n+1}$	Operation
0	0	1	1	Not allowed
0	1	1	0	Set
1	0	0	1	Reset
1	1	Q_n	$\bar{Q}_n$	Hold

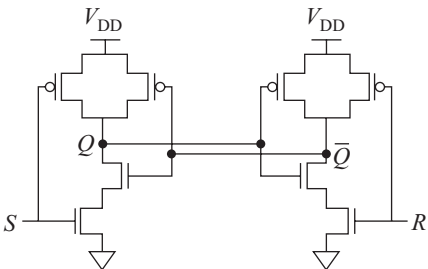


Fig. 6.31 CMOS realizations of NAND-based SR latch circuit

6.10.3 Design of Clocked Latch and Flip-Flop Circuit

The SR latch circuits discussed in Section 6.10.1 are asynchronous in nature in which the output changes as the input logic level changes. In a design, the inputs might come from different paths encountering different path delays. Hence, depending on which input comes first, the output might change accordingly, which might not be the desirable output. To avoid this problem, a clock input is added such that the output changes only in the active period of the clock signal. Generally, the clock signal is a periodic square waveform applied simultaneously to all the gates in the system.

Figure 6.32 shows the gate-level schematic of the clocked NOR-based SR latch.

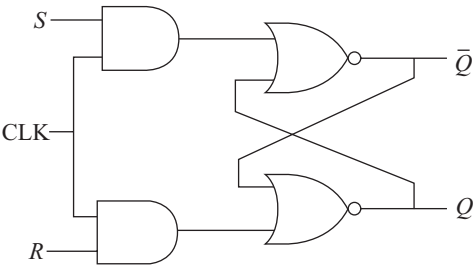


Fig. 6.32 Schematic of the clocked NOR-based SR latch circuit

Figure 6.33 shows the CMOS design of the NOR-based SR latch circuit.

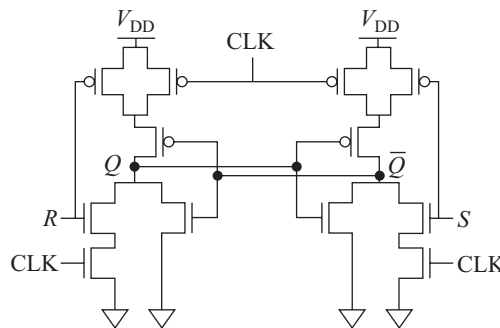


Fig. 6.33 CMOS realization of the clocked NOR-based SR latch circuit

The NAND-based SR latch can also be realized with clock input. This is left as an exercise to the readers.

6.10.4 Design of Clocked JK Latch

The JK latch is commonly known as JK flip-flop. It has three inputs: J, K, and CLK; and two complementary outputs: Q and $\bar{Q}$. JK flip-flop has no not-allowed inputs, unlike SR latches. It allows all possible input combinations. The truth table is shown in Table 6.6.

Table 6.6 Truth table of JK flip-flop

J	K	Q_n	$\bar{Q}_n$	Q_{n+1}	$\bar{Q}_{n+1}$	Operation
0	0	0	1	0	1	Hold
		1	0	1	0	
0	1	0	1	0	1	Reset
		1	0	0	1	
1	0	0	1	1	0	Set
		1	0	1	0	
1	1	0	1	1	0	Toggle
		1	0	0	1	

Figure 6.34 shows the gate-level design of the JK flip-flop.

From the gate-level design of JK flip-flop shown in Fig. 6.34, we can write the expressions for the output as

$$\begin{aligned}\bar{Q} &= \bar{S} + \bar{Q} = \bar{Q} \cdot J \cdot CLK + \bar{Q} \\ Q &= R + \bar{Q} = \bar{Q} \cdot K \cdot CLK + \bar{Q}\end{aligned}$$

Therefore, to implement the PDN for implementing the logic for Q , we require three nMOS transistors connected in series with one nMOS connected in parallel. The inputs to the series connected nMOS transistors are Q , K , and CLK and the input to the parallel connected nMOS transistor is $\bar{Q}$.

Similarly, to implement the PDN for implementing the logic for $\bar{Q}$, we require three nMOS transistors connected in series with one nMOS connected in parallel. The inputs to the series connected nMOS transistors are $\bar{Q}$, J , and CLK and the input to the parallel connected nMOS transistor is Q .

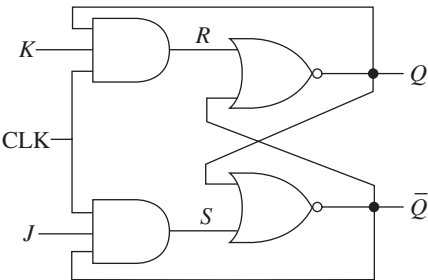


Fig. 6.34 Gate-level design of JK flip-flop

Figure 6.35 shows the CMOS design of the JK flip-flop.

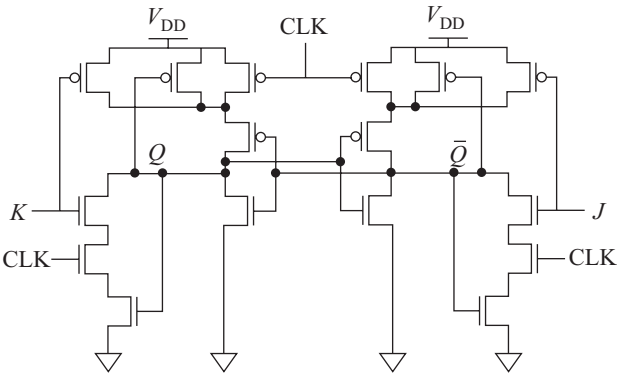


Fig. 6.35 CMOS design of JK flip-flop

6.10.5 Design of CMOS D Flip-Flop

D flip-flop can be designed from the JK flip-flop by connecting an inverter between the J and K inputs. The truth table of D flip-flop is shown in Table 6.7.

Table 6.7 Truth table of D flip-flop

D	Q	Q_n	Q_{n+1}	$\bar{Q}_{n+1}$	Operation
0	0	1	0	1	Hold
0	1	0	0	1	Reset
1	0	1	1	0	Set
1	1	0	1	0	Hold

When the CLK is at logic 1, the D input is propagated to Q output. When the CLK is at logic 0, the Q output is hold. The CMOS design of D flip-flop is shown

in Fig. 6.36. When CLK input is at logic 0, the circuit simply works as two cross-coupled inverter circuits and therefore it holds its outputs. When the CLK input is set to logic 1, depending on the data input D, the output is either set or reset as illustrated in the truth table.

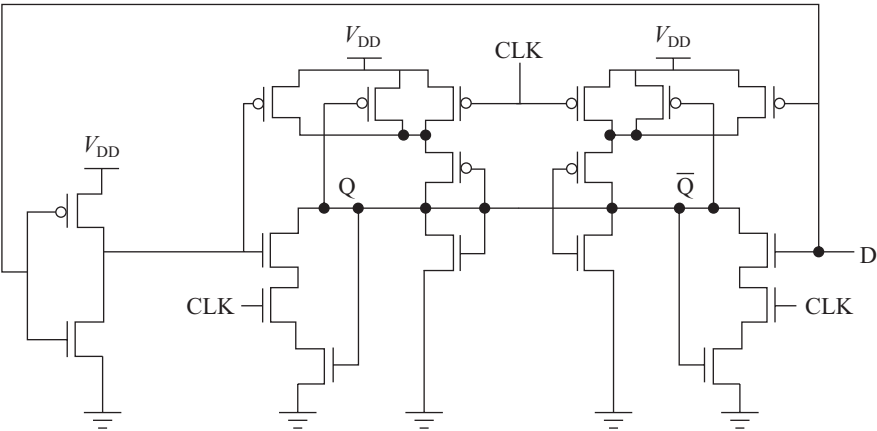


Fig. 6.36 CMOS D flip-flop

6.11 Pseudo-nMOS Logic

In the pseudo-nMOS logic, the PDN is realized by a single pMOS transistor. The gate terminal of the pMOS transistor is connected to the ground. It remains permanently in the ON state. Depending on the input combinations, output goes low through the PDN. An inverter realized using pseudo-nMOS logic is shown in Figs 6.37(a)–(c). The PDN is realized using the process described in Section 6.4.

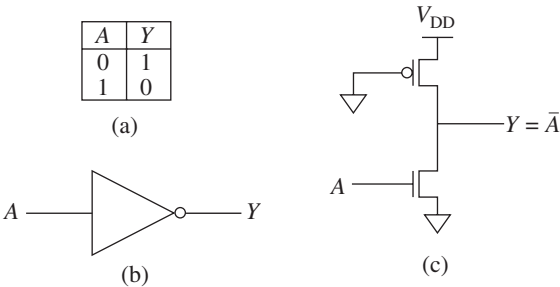


Fig. 6.37 Inverter: (a) truth table; (b) symbol; (c) pseudo-nMOS realization

Example 6.4 Design an XNOR gate using pseudo-nMOS logic.

Solution The pseudo-nMOS realization of two-input XNOR gate is shown in Fig. 6.38.

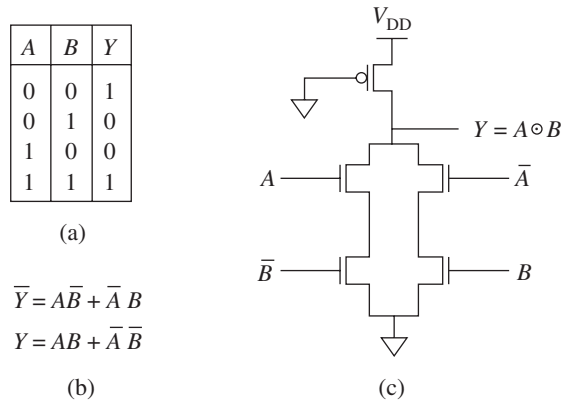


Fig. 6.38 XNOR gate: (a) truth table; (b) Boolean expression; (c) pseudo-nMOS realization

6.12 CMOS Transmission Gate

CMOS transmission gate (TG) is a parallel connection of nMOSFET and pMOSFET that realizes a simple switch. The inputs to the gate of the nMOSFET and pMOSFET are complementary to each other. It is also known as *pass gate*. Figure 6.39 shows a TG.

When signal C is high, $\bar{C}$ is low, thus both nMOS and pMOS transistors are ON, and the nodes A and B are short-circuited. So the input logic is transferred to the output. When signal C is low, $\bar{C}$ is high, thus both nMOS and pMOS transistors are OFF, and the nodes A and B are open-circuited, this is called high-impedance state.

Using CMOS, TG logic circuits can be designed, and it might require lesser number of transistors as compared to the standard CMOS design. This will be clear as we go through some examples in Section 6.12.1.

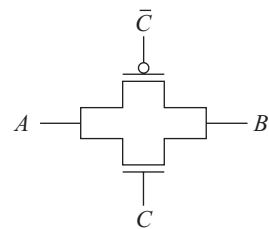


Fig. 6.39 CMOS transmission gate

6.12.1 Design of Combinational Logic Circuits Using CMOS TGs

Example 6.5 Design a 2:1 multiplexer.

Solution Figure 6.40 shows the design of a two-input multiplexer circuit.

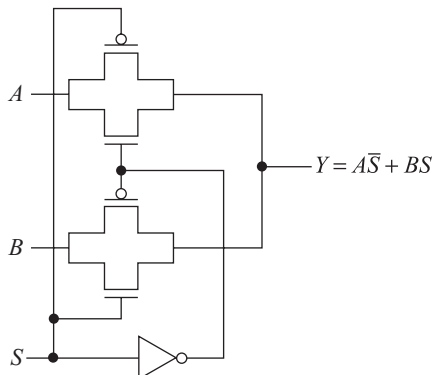


Fig. 6.40 Design of two-input MUX using CMOS TGs

Example 6.6 Design a two-input XOR gate.

Solution Figure 6.41 shows the design of a two-input XOR gate.

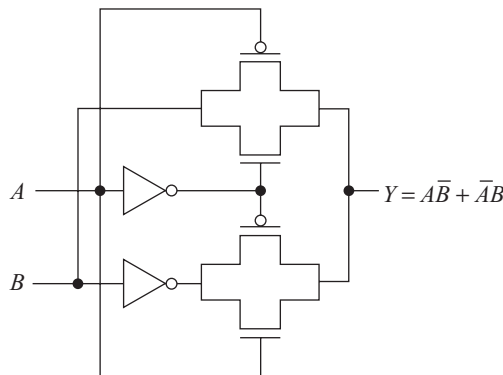


Fig. 6.41 Design of XOR gate using CMOS TGs

Example 6.7 Design of a Boolean function $F = ABC + \bar{A}C + B\bar{C}$.

Solution Figure 6.42 shows the design of a Boolean function $F = ABC + \bar{A}C + B\bar{C}$.

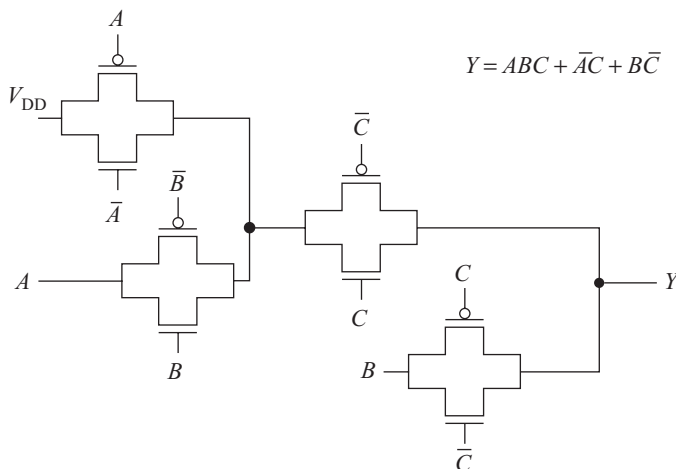


Fig. 6.42 Design of 3-variable Boolean function using CMOS TGs

6.12.2 D-Latch and Edge-triggered Flip-flop

D-latch stores a logic level. This is often called 1-bit register or delay flip-flop. It has single data input D and two outputs: Q and $\bar{Q}$. This is obtained simply by connecting an inverter between the J and K inputs of the JK flip-flop. D input is directly connected to the J input, and the inverted D input is connected to the K input of the JK flip-flop. Block diagram of the D-latch is shown in Fig. 6.43.

Truth table of the D-latch is shown in Table 6.8.

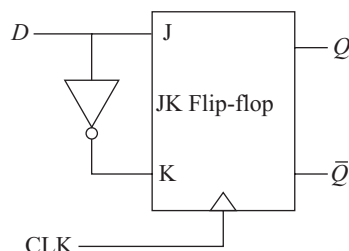
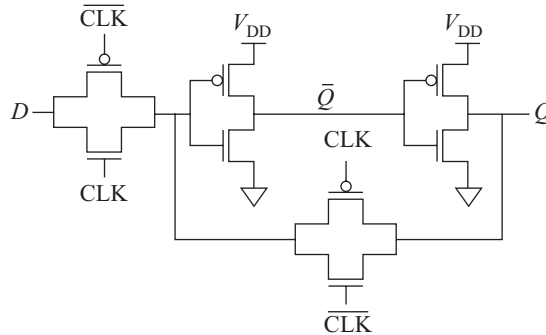


Fig. 6.43 Block diagram of the D-latch

Table 6.8 Truth table of D-latch

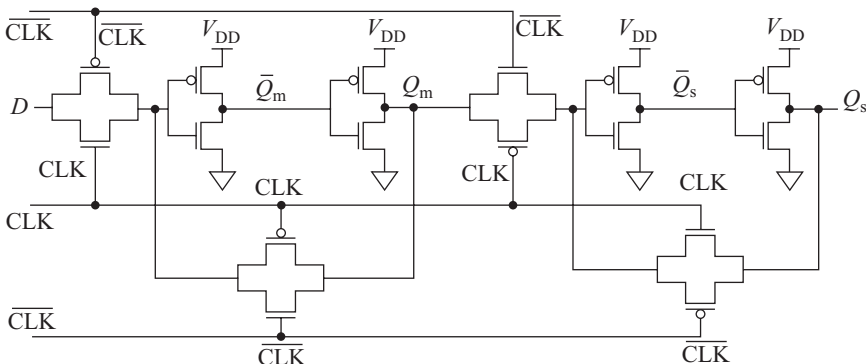
D	Q_{n+1}
0	0
1	1

Figure 6.44 shows the design of the D-latch using CMOS TGs.

**Fig. 6.44** Design of the D-latch using CMOS TGs

The D-latch discussed above is a level triggered flip-flop. This means that the output of the flip-flop changes as long as the clock signal remains at logic high. This has a serious problem in digital logic circuits where output might change during the ON-state of the clock pulse several times if data input changes. But we would like to have output that changes only once in a clock cycle. One way of solving this problem is to design the clock such that it remains at an enabling level for a very short time. In many situations, it may be found that the narrow clock pulses are too narrow to trigger the flip-flop reliably. A better solution is to design the flip-flop such that the output changes only when the clock input makes a transition, rather than at the enabling level of the clock input. One such flip-flop, where output changes only on the clock transitions is a master–slave flip-flop.

Figure 6.45 shows a master–slave flip-flop, which is designed by cascading two D-latches. The clock signal is applied to the first stage (master) and the inverted clock signal is applied to the second stage (slave).

**Fig. 6.45** CMOS edge-triggered master–slave D-flip-flop

6.13 Dynamic CMOS Logic

In the static CMOS logic the functionality of pull-up network is just redundant as the pull-down network alone is sufficient to realize the functionality. In order to reduce the number of transistors, pseudo-nMOS logic is used where pull-up network is replaced by a pMOS transistor with gate terminal connected to the ground. However, pseudo-nMOS logic suffers from large static power dissipation problem.

An alternate method of reducing transistor count is to use dynamic CMOS logic. It is very similar to the pseudo-nMOS logic except one additional nMOS transistor MN connected between the pull-down network and the ground. The pMOS transistor in the PUN and the extra nMOS transistors in the PDN are operated by a clock signal ϕ as shown in Fig. 6.46.

The dynamic logic circuit operates in two phases of a single clock pulse (ϕ). In the pre-charge phase ($\phi = 0$), the output is pre-charged to a logic high level. In the evaluation phase ($\phi = 1$), the output is evaluated based on the applied input logic. Figure 6.47 shows a dynamic CMOS circuit implementing the Boolean function $Y = AB + \overline{A}\overline{B}C$.

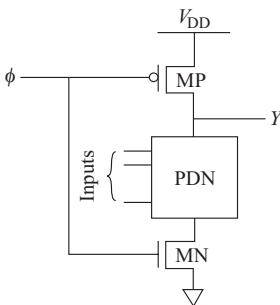


Fig. 6.46 Structure of a dynamic CMOS logic circuit

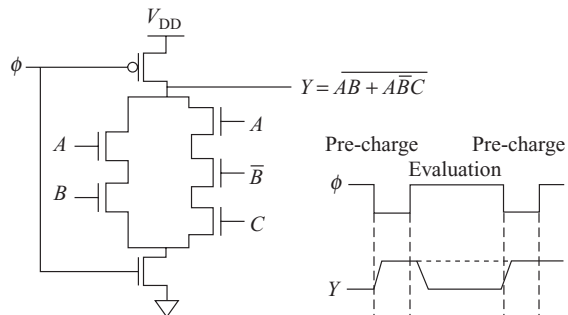


Fig. 6.47 Dynamic CMOS logic circuit

The dynamic logic circuits offer the following advantages over standard CMOS logic circuits:

- Low power dissipation
- Small area due to less number of transistors
- Large noise margin

However, the dynamic CMOS logic circuit has a serious problem when they are cascaded. In the pre-charge phase ($\phi = 0$), output of all the stages are pre-charged to logic high. In the evaluation phase ($\phi = 1$), the outputs of all the stages are evaluated simultaneously. Suppose in the first stage, the inputs are such that the output is logic low after the evaluation. In the second stage, the output of the first stage is one input, and there are other inputs. If the other inputs of the second stage are such that output of it discharges to logic low, then the evaluated output of the first stage can never make the output of the second stage logic high. This is because, by the time the first stage is being evaluated, output of the second stage is discharged, since evaluation happens simultaneously. Remember that the output cannot be charged to logic high in the evaluation phase ($\phi = 1$, pMOSFET in PUN is OFF), it can only be retained in the logic high depending on the inputs.

6.14 Domino CMOS Logic

In order to avoid the problem of dynamic CMOS logic discussed in the previous section, domino CMOS logic is proposed. Domino CMOS logic is a slightly modified version of the dynamic CMOS logic circuit. In this case, a static inverter is connected at the output of each dynamic CMOS logic blocks. The addition of the inverter solves the problem of cascading of dynamic CMOS logic circuits. Figure 6.48 shows a cascaded domino CMOS logic circuit.

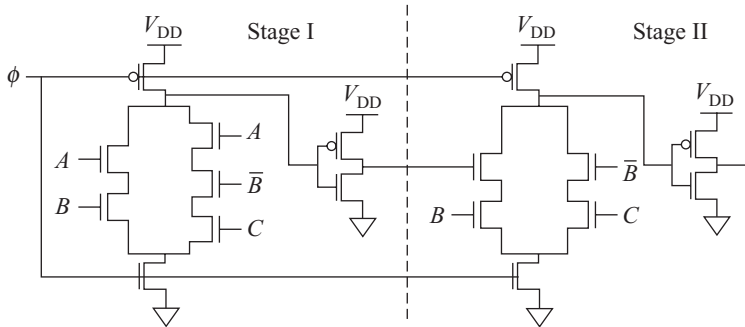


Fig. 6.48 Cascaded Domino CMOS logic circuit

In the pre-charge phase ($\phi = 0$), the outputs of the dynamic CMOS logic circuits are pre-charged to logic high and the output of the static inverter is logic low. In the evaluation phase ($\phi = 1$), outputs of the dynamic CMOS logic circuits can either go to logic low or remain at logic high. Consequently, the output of the static inverter can make only a 0 $\rightarrow$ 1 transition in the evaluation phase. So, irrespective of the input logic, the output of the static inverter cannot make a 1 $\rightarrow$ 0 transition in the evaluation phase.

Domino CMOS helps in reducing the number of transistors as compared to the static CMOS logic. In static CMOS logic we require $2N$ transistors to implement an N -input logic function. However, in dynamic CMOS logic we require only $N + 2$ transistors and for Domino CMOS logic two additional transistors are required. It also solves the cascading problem of dynamic CMOS logic. But it is suitable only for non-inverting logic (the expressions having no complement over whole expression). For inverting logic the expression must be reorganized (to remove the complement over whole expression) before it can be realized using domino CMOS logic.

Example 6.8 Design the following logic using domino CMOS logic and compare it with the static CMOS design.

$$Y = AB + CD + EFG$$

Solution The static CMOS logic design for the given function will be same as shown in Fig. 6.17 except two additional transistors required for inverting the output. This requires $14 + 2 = 16$ transistors. The domino CMOS logic design is shown in Fig. 6.49 which requires only 11 transistors.

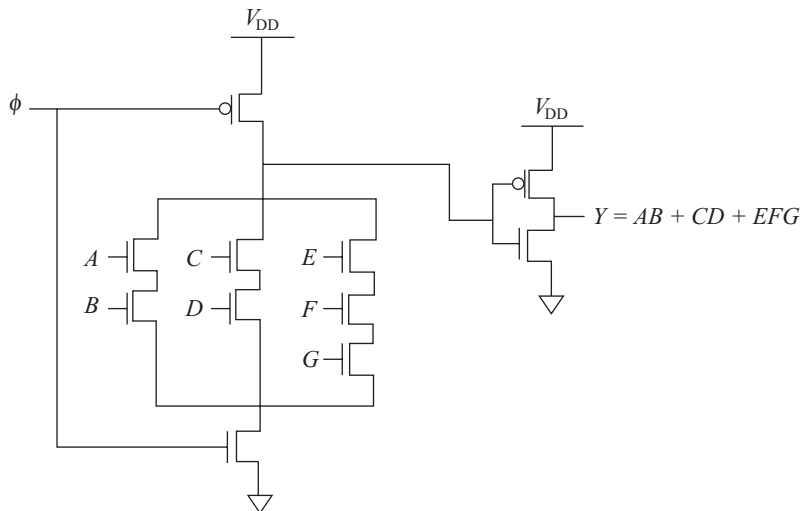


Fig. 6.49 Domino CMOS logic circuit realizing function $Y = AB + CD + EFG$

6.15 NORA CMOS Logic

The dynamic CMOS logic can also be realized using a pMOS transistor to implement the logic through PUN. The PDN is realized using a single nMOS transistor operated by inverted clock ($\bar{\phi}$). When $\phi = 0$, $\bar{\phi} = 1$, the nMOS is ON and the output is pre-discharge to 0 V.

NORA stands for NO Race. The NORA logic style is a modified version of the domino CMOS logic. In this logic style the static CMOS inverters are replaced by the dynamic logic circuits using pMOSFETs. The pMOSFET-based dynamic circuit is operated using inverted clock ($\bar{\phi}$). Unlike nMOSFET-based dynamic circuits, it is pre-charged when $\phi = 1$ and is evaluated when $\phi = 0$. An example of this logic is shown in Fig. 6.50.

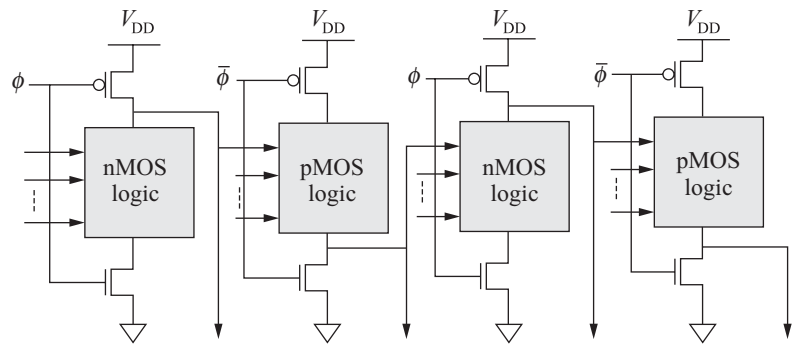


Fig. 6.50 NORA CMOS logic

When clock pulse $\phi = 0$, the output nodes of the nMOSFET logic circuits are pre-charged to V_{DD} through the pMOSFETs and the output nodes of the pMOSFET-based logic circuits are pre-discharged to 0 V through the nMOSFETs.

When clock pulse makes a low-to-high transition, all the nMOSFET-based and pMOSFET-based logic circuits evaluate their outputs depending on the input signals.

NORA logic has a number of advantages over the domino CMOS logic. Firstly, the static CMOS inverter is not required in this logic which saves the chip area. Secondly, NORA logic circuits can be connected in cascade with alternate ϕ -based and $\bar{\phi}$ -based networks.

The NORA logic circuit has a major problem of low-noise margin. The floating output is susceptible to pick up noise due to coupling and charge sharing. That is why this logic is not very popular.

6.16 Zipper CMOS Logic

In a Zipper CMOS logic, a slightly different clocking scheme is used for the pre-charge (ψ_1) and pull-down (ϕ) transistors in nMOSFET-based logic circuits, and $\bar{\phi}$ and ψ_2 for the pull-up and pre-discharge transistors in pMOSFET-based logic circuits. This type of clocking scheme is used in order to keep the pMOSFET pre-charge and nMOS discharge transistors under the subthreshold region. Thus, the transistors operating under subthreshold region conduct weakly to compensate the charge leakage and charge sharing problems. An example of such logic style and the clocking scheme is shown in Figs 6.51(a) and (b).

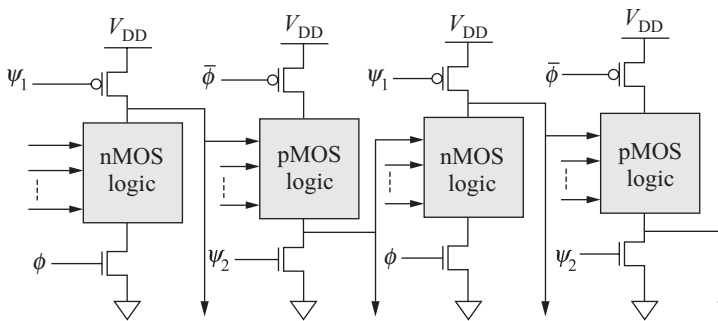


Fig. 6.51(a) Zipper CMOS logic

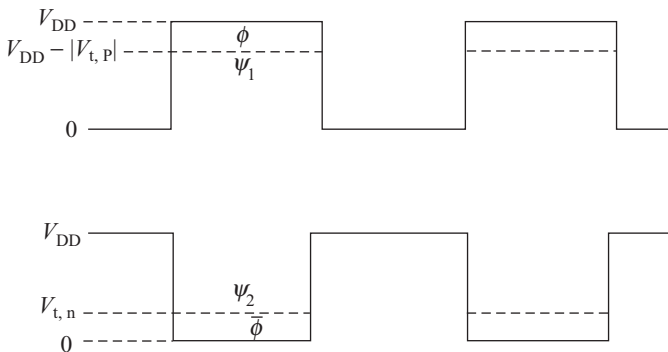


Fig. 6.51(b) Clock signals for zipper CMOS logic

6.17 True Single-phase Clock (TSPC) Dynamic CMOS Logic

The two-phase clocking logic circuits explained earlier require two clocks to be non-overlapping. Two clock signals must be routed carefully in order to reduce clock skew. The TSPC logic is proposed by Yuan et al., which eliminates the requirement of two-phase clocking. TSPC logic requires only single phase clocking, as shown in Fig. 6.52.

When $CLK = 1$, TSPC logic simply works as static CMOS logic. But when $CLK = 0$, only the PUN is activated and the PDN is deactivated. Hence, the circuit just holds its output logic level.

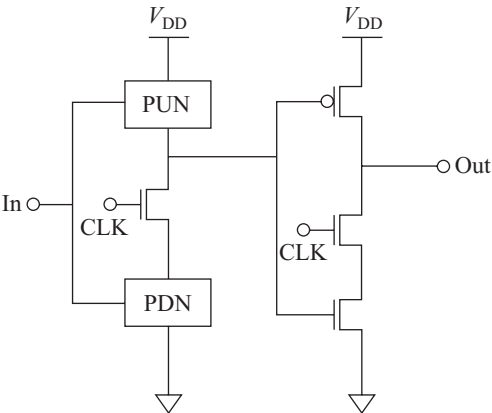


Fig. 6.52 CMOS logic using TSPC logic with latched output

Example 6.9 Design an AND latch circuit using TSPC logic.

Solution An AND latch using TSPC logic can be implemented as shown in Fig. 6.53. The PUN is implemented using two pMOS transistors in parallel, whereas the PDN is implemented using two nMOS transistors in series.

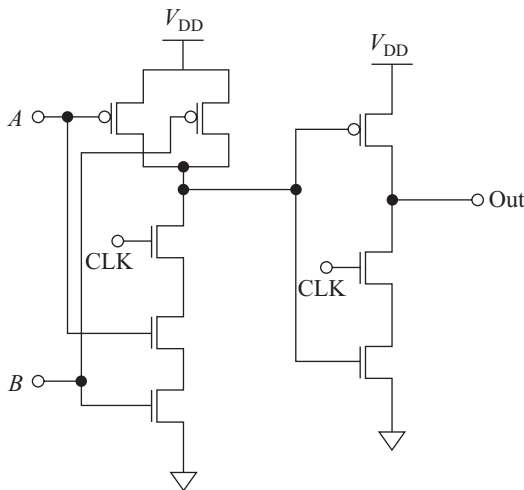


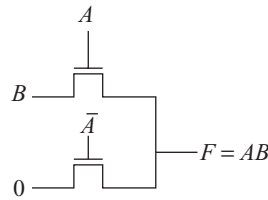
Fig. 6.53 AND latch using TSPC logic

6.18 Pass Transistor Logic

In pass transistor logic, only nMOS transistors are used to design the logic. The inputs signals are applied to both the gate and drain/source terminals. To illustrate the pass transistor logic, let us consider the design of a two-input AND gate. The truth table of two-input AND gate is shown in Fig. 6.54(a).

A	B	$F = AB$	F
0	0	0	When $A = 0$, $F = 0$
0	1	0	
1	0	0	When $A = 1$, $F = B$
1	1	1	

(a)



(b)

Fig. 6.54 (a) Truth table of two-input AND gate; (b) realization of two-input AND gate using pass transistor logic

As seen from the truth table, the function (F) is evaluated for two cases: (i) when $A = 0$, $F = 0$; (ii) when $A = 1$, $F = B$. This logic can be realized using two nMOS transistors connected in parallel as shown in Fig. 6.54(b). When $A = 1$, the upper nMOS is ON, so the output is B ($F = B$). When $A = 0$, the lower nMOS is ON, so the output is 0 ($F = 0$).

This logic is known as pass transistor logic (PTL). This logic uses lesser number of transistors as compared to conventional CMOS logic. The disadvantage of the logic is that the logic high output is degraded as nMOS cannot pass logic high perfectly.

6.18.1 Degradation of Logic High Output Level

The nMOS transistor cannot pass the logic high level perfectly. To analyse this degradation, let us consider a PTL circuit as shown in Fig. 6.55.

The voltage waveforms at nodes In, Out, and x are shown in Fig. 6.56. Clearly, it shows that the logic high output (at node x) of the PTL gate is degraded.

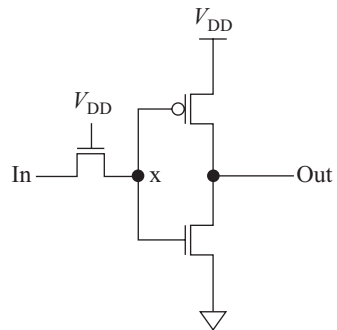


Fig. 6.55 PTL circuit with CMOS inverter at the output

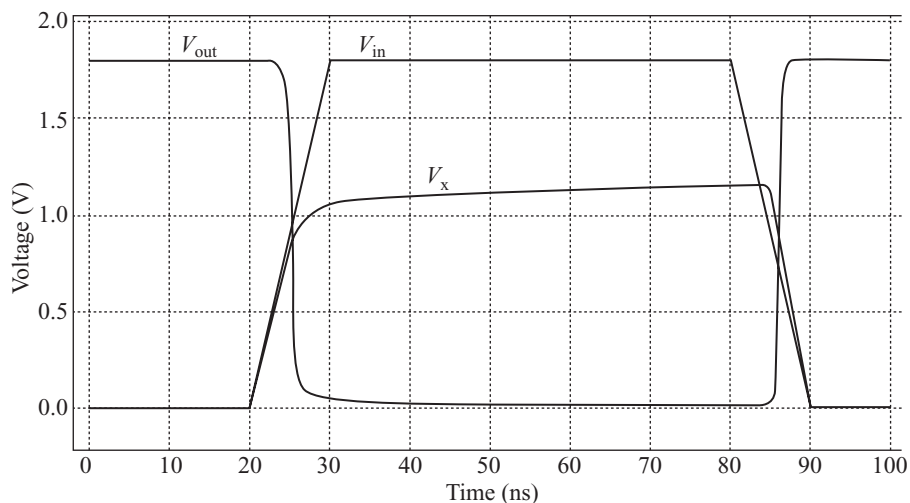


Fig. 6.56 Output waveform of a PTL gate

Due to the logic degradation, the PTL gates cannot be cascaded, as the logic degradation gets carried over several stages, and the final output logic becomes incorrect. To avoid the carry of degradation, a CMOS inverter is connected at the output of the PTL gate as shown in Fig. 6.55.

6.18.2 Design of XOR Gate Using PTL

An XOR gate can be realized using PTL, as shown in Fig. 6.57.

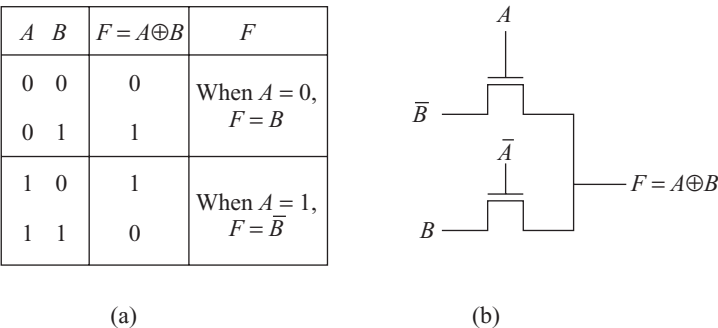


Fig. 6.57 (a) Truth table of XOR gate; (b) XOR gate implemented using PTL

The input and output waveforms of the XOR gate implemented using PTL are shown in Fig. 6.58. It shows that the output waveform is a function of the input transition. For example, when $A = 1$ and $B = 0 \rightarrow 1$, output waveform falls from high to low. Again, when $B = 1$ and $A = 0 \rightarrow 1$, output waveform falls from high to low. But in these two cases, the output waveform is different in shape. Thus, the PTL gate has another issue of having transition-dependent output waveform.

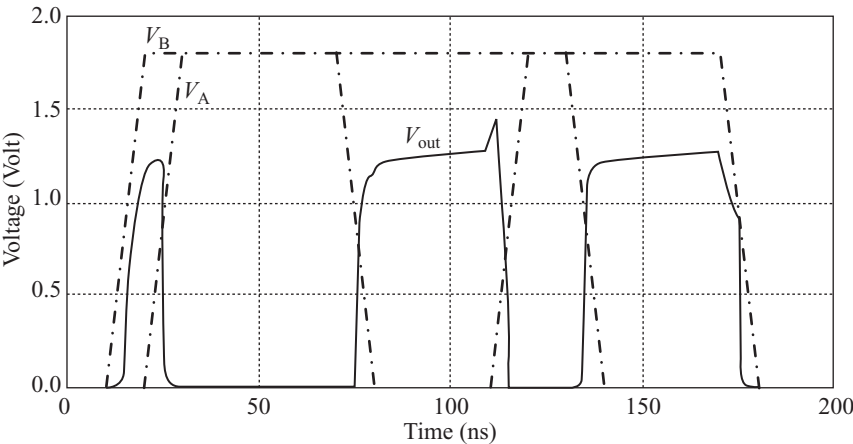


Fig. 6.58 Input-output waveforms of XOR gate using PTL

6.19 Complementary Pass Transistor Logic

In the complementary pass transistor logic (CPL), the inputs are applied both in the true and complement form, and the outputs are also evaluated in both true and complement form, using pass transistor logic. This logic is also known as differential pass transistor logic (DPL). An example of XOR/XNOR is illustrated in Fig. 6.59.

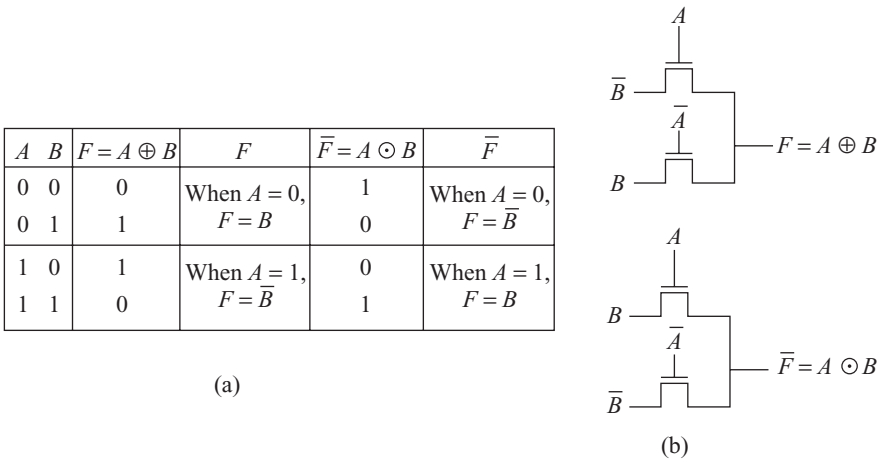


Fig. 6.59 (a) Truth table for XOR/XNOR gate; (b) realization of XOR/XNOR function using CPL

6.19.1 Design of OR/NOR Gate Using CPL

Using CPL, the OR and NOR functions can be implemented as shown in Fig. 6.60. The Boolean expression for an OR gate can be written as

$$F = A + B = A + \bar{A}B = A \times A + \bar{A} \times \bar{B}$$

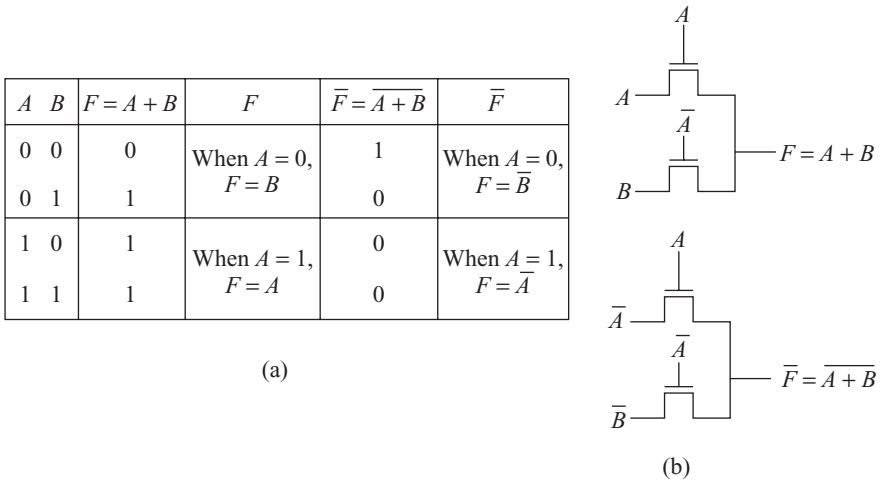


Fig. 6.60 (a) Truth table for OR/NOR gate; (b) realization of OR/NOR function using CPL

Similarly, the Boolean expression for a NOR gate can be written as

$$\bar{F} = \overline{A + B} = \bar{A} \times \bar{B} = A \times \bar{A} + \bar{A} \times \bar{B}$$

6.19.2 Design of AND/NAND Gate Using CPL

Using CPL, the AND and NAND functions can be implemented as shown in Fig. 6.61. The Boolean expression for an AND gate can be written as

$$F = AB = A \times B + A \times \bar{A}$$

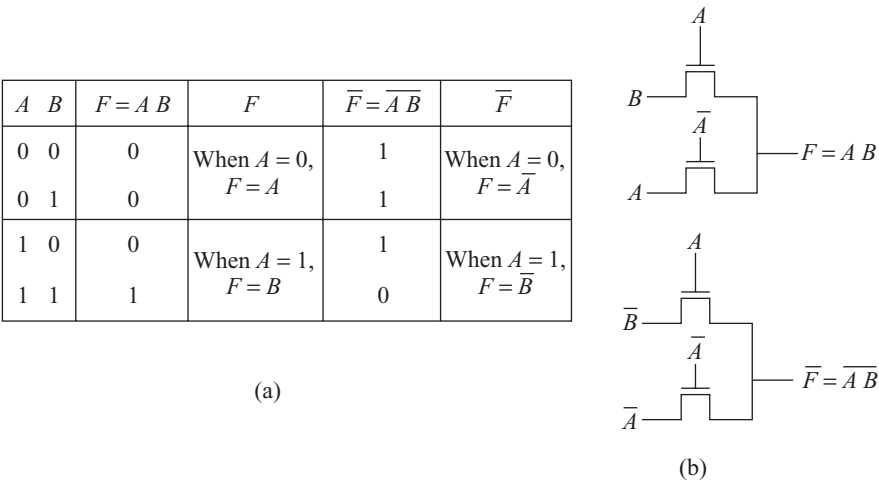


Fig. 6.61 (a) Truth table for AND/NAND gate; (b) realization of AND/NAND function using CPL

Similarly, the Boolean expression for an AND gate can be written as

$$\begin{aligned}\overline{F} &= \overline{AB} = \overline{A} + \overline{B} = \overline{A} + \overline{B} \times (A + \overline{A}) \\ &= \overline{A} + \overline{A} \times \overline{A} + A \times \overline{B} = \overline{A} \times \overline{A} + A \times \overline{B}\end{aligned}$$

6.20 Voltage Bootstrapping

It is a technique to mitigate the problem of threshold voltage drop in digital circuits. The logic degradation occurs in pass transistor logic and enhancement load inverter circuits. To explain the bootstrapping technique, let us consider an inverter circuit with an enhancement-type load as shown in Fig. 6.62(a). When the input voltage (V_{in}) is low, transistor M1 is turned Off. The output voltage (V_{out}) is logic high with a degradation of threshold voltage of the nMOS transistor load and is given by

$$V_{out} = V_x - V_{t,M2} \tag{6.19}$$

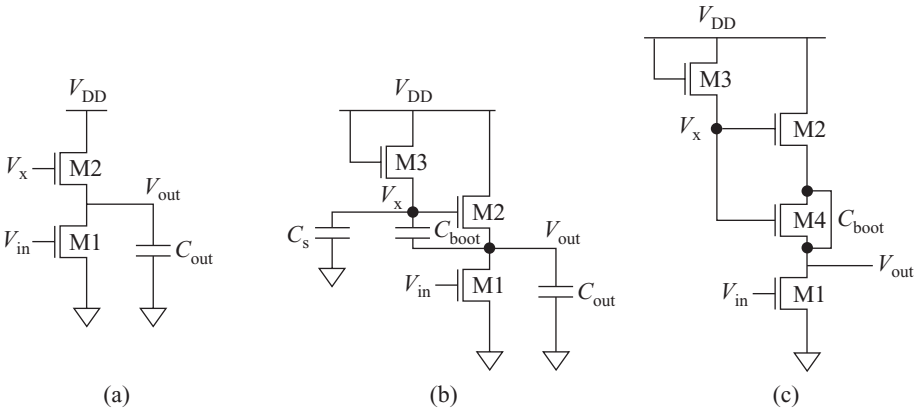


Fig. 6.62 (a) Inverter with enhancement-type nMOS load, (b) dynamic bootstrapping, (c) bootstrapping capacitor realized with a MOS capacitor.

The logic degradation is overcome by increasing the voltage V_x . This technique is illustrated in the circuit shown in Fig. 6.62(b) where two capacitors C_s and C_{boot} are added along with a diode-connected nMOS transistor M3. The node X is connected to ground by C_s whereas it is connected to output by C_{boot} . This configuration increases the voltage V_x as given by

$$V_x \geq V_{DD} + V_{t,M2} \quad (6.20)$$

When V_{in} is high, M1 is turned ON, the output voltage is low given by $V_{out} = V_{OL}$. At this point the voltage V_x is equal to $(V_{DD} - V_{t,M3})$. Total charge on node x can be written as

$$Q_x = C_s (V_{DD} - V_{t,M3}) + C_{boot} (V_{DD} - V_{t,M3} - V_{OL}) \quad (6.21)$$

When V_{in} is low, M1 is turned OFF, and the load capacitor starts charging towards logic high. Now let us consider that V_{out} is fully charged to V_{DD} and at this point the voltage at node x is V_x . So the total charge on node x can be written as

$$Q'_x = C_s V_x + C_{boot} (V_x - V_{DD}) \quad (6.22)$$

During the switching of input from high to low, the total charge at node x cannot change instantaneously. Hence, we can write

$$Q_x = Q'_x$$

$$C_s (V_{DD} - V_{t,M3}) + C_{boot} (V_{DD} - V_{t,M3} - V_{OL}) = C_s V_x + C_{boot} (V_x - V_{DD}) \quad (6.23)$$

Rearranging Eqn (6.23), we can write

$$V_x = (V_{DD} - V_{t,M3}) + \frac{C_{boot}}{C_s + C_{boot}} (V_{DD} - V_{OL}) \quad (6.24)$$

If $C_{boot} \gg C_s$, Eqn (6.24) becomes Eq. (6.25)

$$V_x = 2V_{DD} - V_{t,M3} - V_{OL} \quad (6.25)$$

Equation (6.25) shows that V_x can be increased significantly using the bootstrapping technique. The bootstrapped capacitance (C_{boot}) is realized using an nMOS transistor (M4) with its source and drain connected together as illustrated in Fig. 6.62(c). The capacitance C_s is due to the parasitic capacitances C_{GB} and C_{SB} of M2 and M3 transistors, respectively.

6.21 Differential CMOS Logic

In the differential CMOS logic, the inputs and outputs are to be given and derived both in the true and complement form, respectively connected in parallel. The PUN network is build using only two pMOS transistors. The structure of differential CMOS logic is shown in Fig. 6.63. This logic is also known as cascode voltage switch logic (CVSL) (Kan and Pulfrey1987).

The advantages of differential CMOS logic are as follows:

- As the output is available in both the true and complement form, it eliminates the use of extra inverter circuits where the inputs are required in both true and complement form.

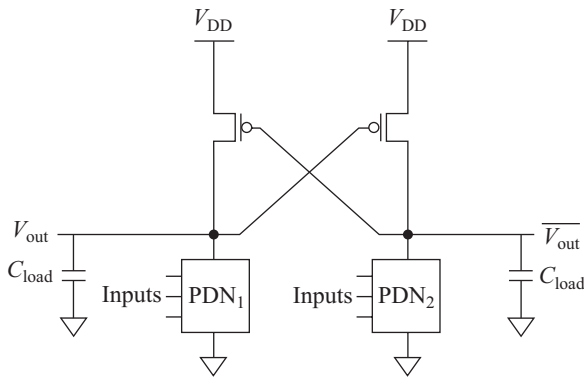


Fig. 6.63 Differential CMOS logic

- It uses just two pMOS transistors in the PUN, instead of using as many pMOS as nMOS transistors used in PDN. This saves a lot of area.

Operation When PDN_1 conducts for some of the input combinations, PDN_2 does not. So the outputs are complementary to each other. The structure of PDN_1 and PDN_2 are dual of each other.

Example 6.10 Design a AND/NAND circuit using differential CMOS logic.

Solution AND/NAND circuit can be realized using differential CMOS logic as shown in Fig. 6.64.

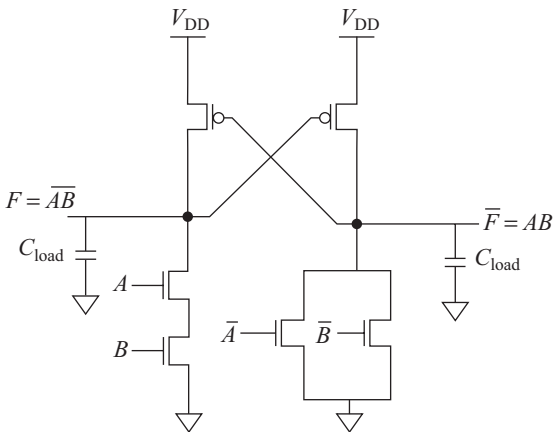


Fig. 6.64 AND/NAND gate realization using differential CMOS logic

6.22 Adiabatic Logic

Adiabatic logic is a special class of logic using the adiabatic concept of thermodynamics for low power VLSI circuits (Chandrakasan and Brdersen 2001). The term adiabatic means there is no heat exchange between a system and the environment. This concept is used for designing adiabatic logic circuits. In this case, the energy stored in the load capacitor during the charging operation is returned to the

power supply during the discharging operation of the load capacitor. To illustrate the logic, let us consider an adiabatic buffer/inverter circuit as shown in Fig. 6.65.

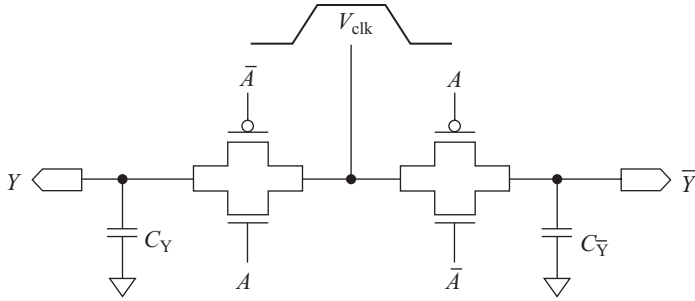


Fig. 6.65 Adiabatic buffer/inverter circuit

In this logic, the V_{DD} is replaced by a power clock which ramps up to V_{DD} with a slope, stays at V_{DD} for some time, and then ramps down to the ground with a slope. When input signal (A) is high, the left transmission gate is ON; hence, the capacitor C_Y charges according to the V_{clk} waveform. So, the output Y is high. The right transmission gate is OFF and the output $\bar{Y}$ is low. While V_{clk} ramps up to V_{DD} , the capacitor C_Y charges up to V_{DD} . After the completion of charging, the output node is at logic high. At this time, only the logic is evaluated and can be applied to the input of similar other gates. When the V_{clk} ramps down to the ground, the capacitor C_Y discharges and returns back the stored charge to the power supply clock. In this way the energy saving is done in the adiabatic logic circuits.

6.23 Dual-threshold CMOS Logic

Generally, a high-speed low-power design requires a reduction in the supply voltage (V_{DD}) and threshold voltage (V_t) of the MOS transistors. But reduction of V_t of the MOS transistor results in high subthreshold leakage problem. In order to avoid the leakage problem, the dual-threshold CMOS logic is used. In this logic, there are two modes of operation: the active mode when the circuit operates to function; and the stand-by mode when the circuit remains in the sleep mode. The general structure of dual- V_t CMOS logic is shown in Fig. 6.66. There are two types of MOS transistors used in this logic: (a) high V_t and (b) low V_t . The low V_t transistors are used to design the core logic that evaluates the function at high speed. When the device operates in the active mode, the stand-by signal is low; hence, the high V_t transistors are ON, and the circuit operates as normal CMOS logic at high speed. But when the circuit does not operate in the stand-by mode, the stand-by signal is high, and the high V_t transistors are OFF. So, they avoid any subthreshold leakage due to the low V_t transistors.

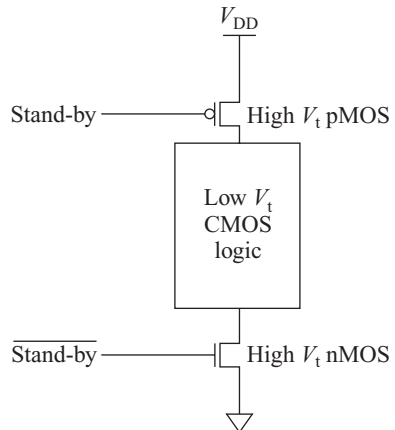


Fig. 6.66 Dual V_t CMOS logic circuit

SUMMARY

- In CMOS digital logic design, all MOSFETs act as switches operating either in cut-off or in saturation region.
- A pMOS passes logic 1 perfectly, but passes a degraded logic 0. On the other hand, nMOS passes logic 0 perfectly, but passes a degraded logic 1.
- In static CMOS logic structure, a pull-up network (PUN) is implemented with pMOS transistors and a pull-down network (PDN) is implemented with nMOS transistors.
- In general, product (AND) terms are implemented by a series-connected nMOS transistors, and sum (OR) terms are implemented with parallel-connected nMOS transistors. Inverse or dual of nMOS network is done in pMOS network.
- In pseudo-nMOS logic, the entire PUN is implemented by a single pMOS transistor with gate terminal connected to ground.
- The dynamic CMOS logic works in two-phases of a single clock. In pre-charge phase ($\phi = 0$), output is pre-charged to logic 1. In the evaluation phase ($\phi = 1$), output is evaluated based on the input signals.
- Domino CMOS logic is obtained by adding a static CMOS inverter at the output of the dynamic CMOS logic after each stage.
- In NORA logic, the static CMOS inverters of Domino logic are replaced by the pMOS dynamic logic, operated with a complemented clock (ϕ) waveform.
- In the differential CMOS logic, both inputs and outputs are in true and complemented form.
- Adiabatic logic and dual-threshold CMOS logic are targeted for low power applications.

SELECT REFERENCES

- Aur, S., D.E. Hecavar, and P. Yang 1987, *Circuit Hot Electron Effect Simulation*, Electron Devices Meeting, 1987 International, Vol. 33, pp. 498–501.
- Bhasker, J. 2003, *A VHDL Primer*, Pearson Education.
- Brown, S. and Z. Vranesic 2002, *Fundamentals of Digital Logic with VHDL Design*, Tata McGraw-Hill, New Delhi.
- Chandrakasan, A.P. and R.W. Brodersen 2001, *Minimizing Power Consumption in CMOS Circuits*, Department of EECS, University of California at Berkeley.
- De Micheli, G. 2003, *Synthesis and Optimization of Digital Circuits*, Tata McGraw-Hill, New Delhi.
- Gerez, S.H. 2007, *Algorithms for VLSI Design Automation*, Wiley.
- Hayes, J.P. 1998, *Computer Architecture and Organization*, 3rd edn, McGraw-Hill.
- Jacob, G.H. 2004, *Design of CMOS Cell Libraries for Minimal Leakage Currents*, Masters Thesis, Computer Science and Engineering Technical University of Denmark, 13 August.
- Kan, M.C. and D.L. Pulfrey 1987, 'A Comparison of CMOS Circuit Techniques: Differential Cascode Voltage Switch Logic Versus Conventional Logic', *IEEE Journal of Solid-State Circuits*, Vol. SC-22, no. 4, August.
- Kang, S.M. and Y. Leblebici 2003, *CMOS Digital Integrated Circuits: Analysis and Design*, 3rd edn, Tata McGraw-Hill, New Delhi.
- Kittel, C. 1996, *Introduction to Solid State Physics*, 7th edn, Wiley.
- Mano, M.M. 2001, *Computer System Architecture*, Prentice-Hall.
- Martin, K. 2004, *Digital Integrated Circuit Design*, Oxford University Press.

- May, G.S. and S.M. Sze 2004, *Fundamentals of Semiconductor Fabrication*, Wiley.
- Rabaey, J.M., A. Chandrakasan, and B. Nikolic 2008, *Digital Integrated Circuits: A Design Perspective*, 2nd edn, Pearson Education.
- Smith, M.J.S. 2002, *Application Specific Integrated Circuits*, Pearson Education.
- Streetman, B.G. 1995, *Solid State Electronics Devices*, 3rd edn, Prentice-Hall of India, New Delhi.
- Sutherland, I., R. Sproull, and D. Harris 1999, *Logical Effort: Designing Fast CMOS Circuits*, Elsevier.
- Taub, H. and D. Schilling 1977, *Digital Integrated Electronics*, McGraw-Hill.
- Weste, N.H.E., D. Harris, and A. Banerjee 2009, *CMOS VLSI Design: A Circuits and Systems Perspective*, 3rd edn, Pearson Education.

EXERCISES

Fill in the Blanks

- _____ can pass a logic 1 perfectly, but cannot pass a logic 0 perfectly.
 - nMOS transistor
 - pMOS transistor
 - CMOS transistor
 - none of these
- _____ can pass a logic 0 perfectly, but cannot pass a logic 1 perfectly.
 - nMOS transistor
 - pMOS transistor
 - CMOS transistor
 - none of these
- Pull-up network (PUN) connects output node to _____.
 - V_{DD}
 - ground
 - input
 - all of these
- Pull-down network (PDN) connects output node to _____.
 - V_{DD}
 - ground
 - input
 - all of these
- AND terms are realized by _____ connections of nMOS in PDN.
 - series
 - parallel
 - cascade
 - anti-parallel

Multiple Choice Questions

- Minimum number of transistors required to design a XOR gate is
 - six
 - eight
 - twelve
 - ten
- Which of the following has minimum propagation delay?
 - ECL
 - TTL
 - RTL
 - DTL
- Which of the following is/are advantages of CMOS?
 - Wide range of supply voltage
 - Greater noise margin
 - Large packing density
 - All of these
- Compared to bipolar technology, output drive current in CMOS is
 - lower
 - higher
 - same
 - dependent upon use
- Pseudo-nMOS logic provides which of the following advantages?
 - Static power dissipation is less compared to CMOS logic.
 - It is much faster compared to other logics.
 - It requires less number of transistors compared to CMOS logic.
 - It is more noise immune.

True or False

1. OR terms are realized by series connections of nMOS in PDN.
2. Two-input NAND gate requires six transistors in static CMOS logic.
3. The AND-OR-INVERT (AOI) gate is suitable for realizing functions in the SOP form.
4. Pseudo-nMOS logic gate is designed using only nMOS transistors.
5. Dynamic CMOS logic operates with two non-overlapping clock pulses.

Short-answer Type Questions

1. Design a CMOS half-adder circuit.
2. Realize a 2:1 MUX using CMOS transmission gate.
3. What is pseudo-nMOS gate and what is its advantage over CMOS gate?
4. Implement the Boolean function $F = \overline{A}(B + \overline{C}D)$ using static CMOS logic.
5. Design the following circuits using transmission gates.
 - (a) D flip-flop
 - (b) Two-input XOR gate
6. Explain why NOR gates are preferred for nMOS circuits, while NAND gates are preferred for static CMOS circuits.
7. Design a static CMOS circuit to implement the following Boolean function:

$$F = \overline{D} \times \overline{E} \times \overline{A} + \overline{B} \times \overline{C}$$

8. Draw the circuit of a CMOS full-adder circuit and explain its operation.
9. Design a clocked inverter using static CMOS logic.
10. Design a two-input XOR gate using pseudo-nMOS logic.
11. Why NOR-based design preferred in pseudo-nMOS logic compared to NAND-based design in CMOS logic?
12. Realize the circuit as shown in Fig. 6.66 using pseudo-nMOS logic.
13. Realize the logic circuit shown in Fig. 6.66 in static CMOS logic.

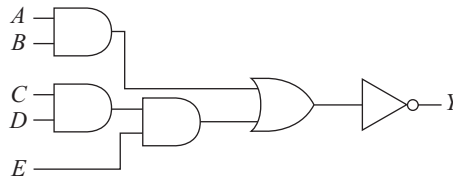


Fig. 6.66 Logic circuit for Questions 12 and 13.

14. Realize the following Boolean function using static CMOS logic:

$$Y = \overline{A} \overline{B} \overline{C} + \overline{B} D + A \overline{C}$$

15. Realize the following Boolean function using pseudo-nMOS logic:

$$Y = \overline{A + B(C + D)}$$

16. Design an even parity generator using static CMOS logic. Assume 3-bit input.
17. Realize the following Boolean function using differential CMOS logic:

$$Y = (A + \overline{B}) C + \overline{A} D$$

18. Design a full-adder circuit using differential CMOS logic.

19. Design the pull down network (PDN) for the following Boolean expression:

$$F = \overline{W + X(Y + Z)}$$

20. Design the pull down network (PDN) for the following Boolean expression:

$$F = \overline{(W + X)(Y + Z)}$$

21. Design the pull up network (PUN) for the following Boolean expression:

$$F = \overline{WX + (Y + Z)}$$

22. Design the pull up network (PUN) for the following Boolean expression:

$$F = \overline{(W + XY + Z)}$$

Long-answer Type Questions

1. Explain the operation of a CMOS inverter with a proper circuit diagram. Draw the inverter characteristic curve and explain the various regions in the curve. What is CMOS noise margin?
2. Design a 3:2 decoder circuit using static CMOS logic.
3. Design a binary-to-grey code converter circuit using static CMOS logic.
4. Design a compressor circuit using static CMOS logic.
5. A logic gate has $V_{OH} = 5$ V, $V_{OL} = 0.2$ V, $V_{IH} = 2.5$ V, and $V_{IL} = 0.8$ V. Calculate the noise margins.
6. In a resistive load inverter, load resistor is 5 k Ω , and the nMOS is modelled by a 200 Ω resistor. Calculate the output voltage when input voltage is high.
7. For $\mu_n/\mu_p = 3.0$, $V_{tn} = 0.6$ V and $V_{tp} = -0.6$ V, find the inverter threshold voltage. Assume $V_{DD} = 1.5$ V.